

Breaking ECDSA with Electromagnetic Side-Channel Attacks: Challenges and Practicality on Modern Smartphones

Felix Oberhansl^{*}, Marc Schink^{*†}, Nisha Jacob Kabacki^{*},
Michael Gruber^{*}, Dominik Klein[†], Sven Freud[†], Tobias Damm[†],
Michael Hartmeier^{*}, Ivan Gavrilan^{*‡}, Silvan Streit^{*‡}, Jonas Stappenbeck^{*}, Andreas Zankl^{*‡}

^{*}Fraunhofer Institute for Applied and Integrated Security (AISEC), Garching, Germany

Email: {felix.oberhansl, marc.schink, nisha.jacob, michael.gruber,
michael.hartmeier, ivan.gavrilan, silvan.streit, andreas.zankl}@aisec.fraunhofer.de

Email (Jonas Stappenbeck): jonas.stappenbeck@tum.de

[†]German Federal Office for Information Security (BSI), Bonn, Germany

Email: {dominik.klein, sven.freud, tobias.damm}@bsi.bund.de

[‡]Technical University of Munich (TUM), Munich, Germany

Abstract—Smartphones handle sensitive tasks such as messaging and payment and may soon support critical electronic identification through initiatives such as the European Digital Identity (EUDI) wallet, currently under development. Yet the susceptibility of modern smartphones to physical side-channel analysis (SCA) is underexplored, with recent work limited to pre-2019 hardware. Since then, smartphone system on chip (SoC) platforms have grown more complex, with heterogeneous processor clusters, sub 10 nm nodes, and frequencies over 2 GHz, potentially complicating SCA. In this paper, we assess the feasibility of electromagnetic (EM) SCA on a Raspberry Pi 4, featuring a Broadcom BCM2711 and a Fairphone 4 featuring a Snapdragon 750G 5G SoC. Using new attack methodologies tailored to modern SoCs, we recover ECDSA secrets from OpenSSL by mounting the Nonce@Once attack of Alam et al. (Euro S&P 2021) and show that a libcrypto countermeasure does not fully mitigate it. We present case studies illustrating how hardware and software stacks impact EM SCA feasibility. Motivated by use cases involving security-critical mobile applications such as the currently developed EUDI wallet, we survey Android cryptographic implementations and define representative threat models to assess the attack. Our findings show weaknesses in ECDSA software implementations and underscore the need for independently certified secure elements (SEs) in all smartphones.

Index Terms—side-channel attacks, smartphones, elliptic curve cryptography

1. Introduction

In today’s world, smartphones have become indispensable in everyday life, playing crucial roles in messaging, payments, and identification, among other sensitive tasks. Their significance continues to grow, highlighted by the European Union’s (EU) plan to introduce a standardized EUDI wallet by 2026 [1]. This demonstrates that smartphones are valuable targets for attackers, and no attack path should be considered irrelevant. Although software exploits, such as Operation Triangulation [2], are usually

the primary concern due to their less stringent threat model, the lack of studies on physical SCA on modern smartphones calls for more exploration in this area. Furthermore, the usage of smartphones for financial transactions, health monitoring, and digital identities (EUDI) warrants the investigation of more powerful adversarial models. Since existing hardware cannot be changed, countermeasures to newly developed hardware attacks must be implemented in software, which is often challenging and insufficient to address the underlying issues.

Physical attacks are directed at the implementation of an algorithm and its physical effects, in our case the EM emissions from smartphones. Such attacks can be advantageous if, for example, cryptographic operations are conducted within a trusted execution environment (TEE) which offers software isolation but does not protect against hardware attacks. In this paper, we show how to recover the secret elliptic curve digital signature algorithm (ECDSA) nonce values from software libraries executed on a modern smartphone by EM SCA.

Attack scope and threat model Our attack scope is to investigate general-purpose application-class processors and popular cryptographic software libraries such as OpenSSL¹ and libcrypto² that may run inside or outside the context of a TEE. We do not investigate tamper-resistant hardware such as SEs. While some flagship and selected mid-range devices include such hardware, many devices rely solely on the TEE. This is particularly relevant as phones are increasingly used for sensitive use cases, leaving devices without an SE to store crucial secrets outside tamper-resistant hardware.

Executing such an attack comes with certain prerequisites: We consider a local, passive EM side-channel adversary with physical access to the smartphone and lab-grade measurement equipment. In order to assess the impact and the limitations of our result for practical smartphone security, we discuss two threat models with respect to these prerequisites in Section 3. In one, the device is screen-locked and the attacker cannot bypass this lock.

1. <https://github.com/openssl/openssl>

2. <https://www.gnupg.org/software/libcrypto/index.html>

In the second model, the attacker can unlock the device, representing (temporary) device theft in combination with weak screen-lock mechanisms. We take security-critical mobile applications, such as the EUDI wallet and its ongoing development as an example throughout this paper in order to discuss the potential real-world impact of our findings.

Related work The practicality of SCA on a smartphone’s application processor is hard to judge from state-of-the-art literature. Given the advancements in SoC technologies and smartphones, it is noteworthy that devices in recent works are more than five years old (Table 1). The authors of [3, 4] demonstrated successful EM attacks on RSA and elliptic curve cryptography (ECC) on smartphones. The Nonce@Once attack [3] recovered ECDSA nonces from conditional swap operations. In [5, 6], non-adjacent form (NAF) and w-ary non-adjacent form (wNAF) ECDSA implementations were investigated on multiple smartphones. In [7], the authors extracted a device unique key from the iPhone 4’s AES engine. The authors of [8] recovered the AES firmware encryption key of the first stage bootloader on an undisclosed mobile device. Apple’s CoreCrypto Library and the ARMv8 Cryptographic Extensions for AES were analyzed on an iPhone 7 in [9]. Challenges when targeting AES with EM SCA on an ARM Cortex-A architecture were described in [10]. They used a Raspberry Pi 4 for their experiments. For EM attacks on older devices we refer the reader to [11, 5, 12] and the survey in [13]. More recent smartphones, such as the iPhone 8 Plus (2017) [14], Xiaomi Redmi 7A (2019) [15], iPhone 13 and 14 Pro (2022) [16], or Google Pixel 9 (2024) and Samsung Galaxy S25 (2025) [17] were only used for forensic SCA, where no cryptographic algorithms were targeted; instead, PIN entry [14], browsing activity [15], differentiation between software activities [16] or display contents [17] were investigated.

Complexity of contemporary smartphones Nowadays, complex SoC platforms are at the heart of smartphones. This complexity impacts the feasibility of SCA. Modern mobile SoCs feature multiple CPUs, often in clusters of two or more. These clusters are optimized either for power consumption, performance, or a trade-off between the two. This allows versatile scheduling options, resulting in a vast evaluation space for EM SCA. The CPU cores can dynamically adapt their clock frequency with the upper limit reaching more than 3 GHz. The integrated circuit (IC) for the smartphone is usually fabricated at advanced semiconductor nodes, such as 10 nm and below. All this potentially increases the challenge for successful EM side-channel measurements.

Contribution Our main contribution is a thorough analysis of the practicality of EM SCA on a contemporary smartphone. Table 1 highlights the gap between smartphones that were targeted in existing literature and the Fairphone 4, our main target device. More precisely, we make contributions in the following areas:

- **Threat models and vulnerability assessment.** We define realistic threat models and provide an overview of how cryptography is used on Android smartphones. We later assess our practical attack

findings in the context of these defined threat models.

- **Methodology.** Starting from the attacks on ECC conditional swap operations in [18, 19, 3] we adapt the attack methodology to reproduce it on more complex devices. Among others, we investigate activity-modulated signals at different frequency bands and methods for leakage assessment and exploitation. We also analyze the countermeasures proposed in [3] and implemented in libgcrpt and come to the conclusion that they offer insufficient protection.
- **Hardware complexity.** We start our investigations with a Raspberry Pi 4 as baseline to evaluate our SCA methodology. Subsequently we investigate the Fairphone 4’s Snapdragon 750G 5G SoC which is also used in phones such as the Samsung Galaxy A52 5G and the Xiaomi Mi 10T Lite. We show the impact that multi-cluster CPUs with high frequencies and dynamic frequency scaling have on the attack.
- **Software complexity.** We evaluate the complexity introduced by the software stack in case studies on the Fairphone 4, where we can install both Android and a native Linux operating system (OS) to which we can easily apply constraints. To the best of our knowledge, we are the first to investigate complete Android apps and the increased complexity this introduces. We demonstrate that the attack is possible despite dynamic scheduling, dynamic frequency scaling and background activities.

Our findings suggest that, while SCA of smartphones is no longer as simple as in [3, 4, 5, 6], attacks are still possible and therefore remain a threat to contemporary smartphones. Thus, the usage of certified secure elements should be mandatory for critical applications.

Code and traces that highlight our methodology are publicly available³. Following coordinated disclosure, we made our findings available to the affected libraries (Appendix B).

2. Background

In this section, we introduce general preliminaries on EM SCA, ECC and ECDSA.

2.1. Electromagnetic side-channel analysis

SCA exploits unintentional leakage (e.g., timing [20], power consumption [21], or electromagnetic emissions [22]) from a system to infer sensitive data. Due to their accessibility, smart cards and other embedded devices are the most investigated targets for these physical side channels. In the past, more complex devices such as PCs or smartphones were mostly investigated for logical, i.e., microarchitectural, side channels such as Spectre [23]. Only recently, the investigation of such devices with regard to EM SCA gained attention [5, 4, 3, 6, 24].

3. <https://github.com/Fraunhofer-AISEC/breaking-ecdsa-emsca-smartphones>

TABLE 1: Smartphones in state-of-the-art literature that were investigated using SCA.

Device	Release	Reference	SoC	Arch.	CPU ^a	Node
iPhone 4	2010	[7]	Apple A4	n/a ^c	n/a ^c	45 nm
iPhone 4	2010	[5]	Apple A4	32-bit	1xA8 @ 800 MHz	45 nm
Xperia X10	2010	[5]	Snapdragon S1	32-bit	1xA5 @ 800 MHz	45 nm
-	2012	[6]	Snapdragon S4	32-bit	2xA5 @ 1.2 GHz	45 nm
„b	2012	[8]	„b	n/a ^c	n/a ^c	32 nm
Galaxy Centura	2013	[4]	Snapdragon S1	32-bit	1xA5 @ 800 MHz	45 nm
iPhone 7	2016	[9]	Apple A10 Fusion	64-bit	2xHurricane @ 2.34 GHz 2xZephyr @ 1.05 GHz	14 nm
Alcatel Ideal	2016	[3, 4]	Snapdragon 210	32-bit	4xA7 @ 1.1 GHz	28 nm
ZTE ZFIVE	2018	[3]	Snapdragon 425	64-bit	4xA53 @ 1.4 GHz	28 nm
Raspberry Pi 4	2019	This Work, [10]	BCM2711	64-bit	4xA72 @ 1.8 GHz	28 nm
Fairphone 4	2021	This Work	Snapdragon 750G 5G	64-bit	2xA77 @ 2.21 GHz 6xA55 @ 1.8 GHz	8 nm

^a All CPUs are ARM Cortex cores.

^b Device not disclosed in paper, release year and semiconductor node were obtained from authors.

^c The attacks do not target the CPU but an internal AES engine.

Profiled SCA is a special case of SCA, which relies on collecting measurement data from a device similar or identical to the target in order to train an attack model during the profiling phase. The adversary requires access to such a device and the ability to run and monitor the cryptographic operations for profiling. In the attack phase, profiled attacks typically require only a small number of traces, making them highly efficient due to the pre-built profiles.

2.2. Electronic identification and ECDSA

Digital signature algorithms (DSAs) ensure the integrity and authenticity of digital transactions. ECDSA uses elliptic curves as a mathematical primitive. It is favored in many applications due to its ability to provide strong security with small key sizes, making it efficient in terms of computational and storage requirements.

DSAs rely on private–public key pairs. The public key is shared with a remote party. The secret, private key remains with the signer. A common use case is electronic identification to a relying party. For example, if Alice wants to authenticate to Bob, he generates a challenge and sends it to Alice. Alice signs the challenge and returns the signature. Bob verifies the signature using Alice’s public key.

For an ECDSA signature to be created, first an elliptic curve in form of its underlying field and equation must be chosen. Each curve has a base point G which generates a cyclic group of order n . The curve’s point at infinity, also called neutral element, $\mathcal{O}$ is $\mathcal{O} = nG$. Points can be represented in projective coordinates $(X : Y : Z)$ or affine coordinates (x, y) .

To sign a message (e.g., challenge), Alice selects a random scalar d_A as her secret private key and the curve point Q_A , obtained by the scalar-by-point multiplication $Q_A = d_A G$, as her public key. A message m is signed by first getting its digest z from a cryptographic hash function. Subsequently, the signer chooses a random nonce k such that $1 \leq k \leq n-1$. The nonce is used as a scalar for a scalar-by-point multiplication that yields the coordinates x and y of a new point on the curve: $(x, y) = kG$. The signature is the tuple (r, s) , where $r = x \pmod n$ and

$s = k^{-1}(z + r \cdot d_A) \pmod n$. The process is repeated for a new random k if either r or s are zero.

To verify the signature, Bob can compute z from the message m and $w = s^{-1} \pmod n$, $u_1 = z \cdot w \pmod n$ and $u_2 = r \cdot w \pmod n$. From this, the candidate coordinates (x', y') can be obtained as $(x', y') = u_1 G + u_2 Q_A$. If $x' \equiv r \pmod n$, Bob accepts the signature.

The nonce k must be secret, random and unique for each new signature that is created. If an adversary knows k , she can recover d_A from the signature (r, s) . We summarize relevant works on SCA of ECDSA in [Section 4](#).

3. Threat models

To evaluate the practicality of our attack and clearly explain the motivation for hardware attacks on smartphones, we introduce two threat models. These models serve primarily to assess how well our analyses, presented in [Section 6](#), align with realistic attacker capabilities in [Section 7](#). We distinguish threat models based on whether the adversary can unlock the phone ([Section 3.1](#)) or not ([Section 3.2](#)).

3.1. EM SCA on unlocked smartphone

In this subsection, we assume that the adversary has an unlocked phone in her possession and user-level access. In this case it might seem questionable whether SCA is needed. The adversary can already access the vast majority of the victim’s data and install malicious software. However, certain cryptographic secrets are not accessible, e.g., secrets stored within a TEE or SE. While the SE typically features hardware countermeasures against physical SCA, the TEE does not, as it uses the same processor as generic software running on the device.

The unlocked phone might allow an adversary to trigger cryptographic processes, as long as no further authentication besides the unlock mechanism is required (see authorization for key usage below). SCA can then be used to recover the involved cryptographic secrets. Depending on how invasive the preparations needed for the EM SCA are, the device could afterwards be returned to its owner without raising suspicion.

To underline the realism of this threat model, we emphasize that it is equivalent to scenarios where the adversary obtains a screen-locked smartphone, but can easily bypass the lock. Recent work has shown that the security of a simple unlock PIN depends on whether countermeasures such as throttling and blocklisting are deployed and designed reasonably [25]. Graphical passwords such as unlock patterns are often biased and offer less security than guessing a three-digit PIN [26]. Additionally, they face the threat of smudge attacks [27], which might apply to PINs as well. Biometric mechanisms such as a fingerprint sensor can be overcome as shown in [28, 29, 30]. Also, facial recognition systems for unlocking a phone are potentially vulnerable to presentation attacks [31]. The practical feasibility of bypassing a screen lock depends on the specific locking mechanism, the user’s vigilance, and the resources an attacker is willing to expend.

It is important to note that Android allows app developers to additionally gate access to cryptographic keys⁴. For example, a phone could be unlocked with a PIN, but usage of a cryptographic key could require additional biometric authentication. Our threat model here assumes the adversary can bypass device unlock and any key-use authorization mechanisms.

Note that for digital signing with non-repudiation, the adversary can also be the owner: Non-repudiation implies that the private key has sufficient protections in place that the owner cannot later repudiate actions she takes with the key. If a private key is compromised, it undermines non-repudiation.

3.2. EM SCA on locked smartphone

If the phone is insurmountably locked, the adversary usually has no possibility to read out data directly. We assume that no pairing via USB, Bluetooth, or Android Debug Bridge (ADB) is possible. Therefore SCA of cryptographic software running both inside and outside the context of a TEE is a reasonable path of attack. SCA on locked smartphones can be further divided into two scenarios. A scenario where the device is (temporarily) stolen and one where the owner always remains in possession of the device.

If the owner remains in possession of the smartphone, attacks usually must not require any noticeable physical modifications to the device and must be carried out within a short period of time. The adversary could use a probe placed on the display or the device’s casing (e.g., by hiding it in a desk), as in [3, 4, 6, 5] or hidden in a charger [5].

If the device is stolen, the applicability of attacks is reduced to scenarios, where cryptographic operations are triggered by normal system activity (e.g., decryption of incoming messages or TLS handshakes). This might require notifications to be enabled for the locked phone. The Android auto-reboot feature [32] could complicate this threat model. Devices for which this feature is enabled and which are not unlocked for three days are automatically rebooted. After the reboot, disk encryption might be active and the device is set into the before first unlock (BFU) state, which might disable certain unlock

methods and features such as the automatic decryption of incoming messages. However, this does not mitigate the attack, as the targeted cryptographic routine is likely to be triggered at least once in this time span. For messaging, an adversary could send a message to the victim to induce execution.

4. Cryptography on Android: Implementations and attack surfaces

In this section, we dive deeper into the potential vulnerability of smartphones to EM SCA by investigating how cryptography can be implemented on Android smartphones. We review considerations for the EUDI wallet and provide an overview of SCA targeting ECDSA.

4.1. Cryptography on Android smartphones

We focus our investigation on Android smartphones due to the transparency and openness of the Android ecosystem and its wide deployment. We emphasize that this openness does not imply less security compared to other platforms. The Android Keystore [33] is the default provider for cryptography on Android smartphones. For compatibility and the support for algorithms not included in the Android Keystore, most phones also have alternative providers such as OpenSSL or Bouncy Castle⁵. The Android Keystore is the only possibility to use a TEE or StrongBox, the latter being Android’s term for an SE, i.e., dedicated tamper-resistant hardware.

The Android Device Security Database⁶ provides an overview of security features available on phones, e.g., which devices include StrongBox and which do not [34]. High-end devices such as Google Pixel phones and the Samsung Galaxy S series support StrongBox. However, even if StrongBox is available, there are important caveats, which might block its wide adoption for applications. For one, TEE is the general default for the Android Keystore and developers are urged to only use StrongBox for applications requiring the highest level of security [33]. Since StrongBox is typically implemented outside of the SoC, within a dedicated IC, operations are much slower and more limited with regards to concurrency than if the TEE were used [33]. Therefore, an app must explicitly request StrongBox-backed keys if it shall be used over the TEE. Popular messaging apps like Signal⁷, Telegram⁸ and Threema⁹ do not do this, since the performance requirements for messaging are demanding with regard to the amount of data that needs to be encrypted. Second, not all algorithms are necessarily supported by StrongBox. We read out the supported algorithms on a Google Pixel 8, which features the Titan M2 SE [35]. Of the ECC signature algorithms, only the *secp256r1* curve is supported by StrongBox.

Furthermore, not all modern smartphones are equipped with the StrongBox feature. Even recent mid-range devices such as the Samsung Galaxy A55 or the Sony

4. <https://source.android.com/docs/security/features/authentication>

5. <https://www.bouncycastle.org/>

6. <https://www.android-device-security.org>

7. <https://github.com/signalapp/Signal-Android>

8. <https://github.com/DrKLO/Telegram>

9. <https://github.com/threema-ch/threema-android>

Xperia 10 do not include suitable hardware for StrongBox yet. Of the twelve devices with a release after January 2022 that are listed in the database of [34], only five support StrongBox, while all of them support TEE. In fact, all devices in the database dating back to January 2018 support TEE. Thus, many smartphones will continue to rely on software-implemented cryptography on non-tamper-resistant hardware in the near future.

Security-critical mobile applications must store sensitive data, e.g., keys for electronic identification (Section 2). We use the EUDI wallet, which is currently being developed, to highlight the dilemma that arises when devices without secure hardware must store highly sensitive data. Electronic identification is often built around DSAs such as ECDSA (Section 2.2). Within the wallet, the wallet-secure cryptographic device (WSCD) is the module in charge of managing the secret wallet keys for its holder. The EU requires all WSCDs to be tamper-resistant and certified. The draft of the implementation act [36] lists the Common Criteria (CC) assurance level EAL4+ and the AVA_VAN.5 vulnerability assessment [37] as potential certification targets. This demonstrates a strong commitment to the wallet’s resilience, also with regard to hardware attacks. Furthermore, it underlines the necessity to investigate the susceptibility of smartphones to hardware attacks.

While there are smartphones that could potentially fulfill such a requirement (e.g., Google Pixel phones which include a Titan M2 [35, 38]), the EUDI wallet also plans to consider older smartphones and cheaper models lacking secure hardware [39]. According to the EUDI documentation, such devices shall rely on a remote hardware security module (HSM) to act as WSCD [39]. In this case, the wallet keys are protected, but the wallet app must authenticate itself to the remote HSM, and the secret authentication key must be securely stored on a smartphone lacking secure hardware.

Without StrongBox, TEE-based implementations are the next best option for the EUDI wallet app. TEEs cannot be considered secure against hardware attacks, i.e., tamper-resistant. For example, the Qualcomm TEE v5.8 on the Snapdragon 865 passed an EAL2+ evaluation [40], which does not consider hardware attacks [37]. Therefore, a TEE-based authentication of the wallet app to the remote WSCD would be an attractive target for EM SCA and our work aims to provide an objective quantification of the risks associated with such attacks.

4.2. SCA of ECC implementations

Within our work, we target open-source libraries and their ECDSA implementations. As discussed above, Android apps should use the Android Keystore for cryptography. However, there are multiple reasons to focus on libraries such as OpenSSL and libcrypto first. For one, the implementation details of StrongBox and TEE, the default backends of the Keystore, are secret. It therefore makes sense to start by examining a known implementation. As far as TEEs are concerned, the use of countermeasures is limited to software, as is the case in open-source libraries. Second, libraries such as OpenSSL are also available as an alternative provider to the Keystore on most Android devices. Finally, some apps ship their own

libraries to ensure compatibility across devices or utilize specific features absent in the Keystore. The Telegram Messenger, for example, relies on its own BoringSSL implementation [41].

The selection of ECDSA as targeted signature algorithm is straightforward. It is a mandatory algorithm for the Android Keystore and a likely choice for applications such as the EUDI wallet.

In this section, we review state-of-the-art side-channel attacks on ECC, most notably attacks on the conditional swap operation [3, 18, 19].

Most SCA of ECC focus on the scalar-by-point multiplication. This is because it is a critical operation that involves either ephemeral keys, often referred to as nonces, that allow to recover the long-term keys, or long-term secrets directly (Section 2.2).

Algorithm 1 Double-and-always-add multiplication.

Input: n -bit scalar $k = k_{n-1}, k_{n-2}, \dots, k_0$; elliptic curve point P .

Output: $R \leftarrow [k]P$.

```

1:  $R \leftarrow \mathcal{O}$  ▷ Neutral element
2: for  $i \leftarrow n - 1$  to  $0$  do
3:    $R \leftarrow 2R$ 
4:    $T \leftarrow R + P$ 
5:    $R, T \leftarrow \text{CT\_SWAP}(R, T, k_i)$  ▷ Swap if  $k_i = 1$ 
6: end for
```

Attacks on the conditional swap operation The double-and-always-add algorithm and the Montgomery ladder are two popular choices for implementing a constant-time scalar-by-point multiplication for Weierstrass and Edwards curves [42]. The former is shown in Algorithm 1 and used by libraries such as libcrypto. OpenSSL uses a Montgomery ladder [43], which differs slightly from the procedure in Algorithm 1. The most relevant difference for this work is that the swap operation is either required twice per iteration of the for loop or the swap condition *cond* is merged to $\text{cond} = k_i \oplus k_{i+1}$. In both cases, the sequence of double-and-add operations is constant. This is achieved by conditionally swapping R and T . The constant-time conditional swap operation CT_SWAP (Algorithm 2) can be exploited for SCA. This was demonstrated previously on microcontrollers [18, 19] and smartphones [3]. It is noteworthy that popular SCA countermeasures for ECC such as coordinate re-randomization, point blinding and scalar blinding are not or only partially effective against the attack. We discuss countermeasures in more detail in Section 5 and Section 7.

The swap condition is related to a single bit of a secret scalar, which is either one, i.e., the points are swapped, or zero, i.e., the points are not swapped. Algorithm 2 shows how an arithmetic mask can be used to achieve a constant runtime independent of the swap condition. The authors of [3] discussed that this comes at the cost of EM leakage amplification. First, the single bit in *cond* is expanded to either an all-zeros or all-ones mask. Further, δ is either all-zeros or a machine word that holds a pseudo-random value with a Hamming weight of approximately half the machine word size. When $a[i]$ and $b[i]$ are written back to memory, the Hamming distance of the old and new

Algorithm 2 Constant-time conditional swap operation CT_SWAP.

Input: Two arrays a, b of size n machine words, representing a coordinate of an elliptic curve point; Swap condition $cond \in \{0, 1\}$.

Output:

If $cond = 1$: $a \leftarrow b, b \leftarrow a$;

If $cond = 0$: $a \leftarrow a, b \leftarrow b$.

```

1:  $mask \leftarrow 0 - cond$   $\triangleright mask$  is all-ones or zeros
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $\delta \leftarrow (a[i] \oplus b[i]) \& mask$ 
4:    $a[i] \leftarrow a[i] \oplus \delta$ 
5:    $b[i] \leftarrow b[i] \oplus \delta$ 
6: end for
```

value is either zero or also corresponds to approximately half the machine word size. The computation of δ and the store operations are repeated for all n machine words. Moreover, CT_SWAP is executed for each coordinate of the elliptic curve point. Therefore, the leakage amplification depends also on the used curve. For 64-bit architectures and the *secp521r1* curve, this amounts to the projective coordinates X , Y and Z , each of which is stored in nine machine words and includes additional flags such as *negative*, which is part of OpenSSL’s *bignum* representation. For *Ed25519*, only four machine words per coordinate are necessary.

After the publication of [3], libcrypto adopted a countermeasure, which we analyze in detail in Section 5. Adopting a countermeasure for OpenSSL was discussed but never realized^{10,11,12}. Therefore, it is still vulnerable to the attack from [3, 18, 19].

Alternative implementations and attacks Alternative attacks on the double-and-always-add routine do not target the conditional swap, but the value of an intermediate coordinate by using online template attacks to recover whether an addition was executed on the input coordinate or not [44, 45, 46]. Some libraries such as BoringSSL and WolfSSL use a windowed multiplication with a precomputed table [47]. Profiled and non-profiled attacks on such implementations were proposed in [48, 49]. The authors of [50] proposed a template attack that targets intermediate points on a curve to recover the first few bits of the nonce. This could in theory work independently of the used multiplication algorithm. Lastly, many libraries formerly used wNAF and NAF double-and-add algorithms¹³, where the sequence of double-and-add operations leaks nonce bits. This was exploited in [6, 5, 24].

Target software The attacks from [3, 18, 19] are the basis for our work. In the following, we focus mainly on OpenSSL (v3.6.0) and the *secp521r1* Weierstrass curve. Additionally, we investigate libcrypto (v1.9.4), which adopted an EM SCA countermeasure after publication of [3], and *Ed25519*. It should be noted that while both OpenSSL and libcrypto may choose to integrate coun-

termeasures against EM SCA, they do not consider such attacks in their threat models.

5. Revisiting Nonce@Once

In this section, we explore alternative methodologies for SCA of the conditional swap operation in ECDSA [3, 18, 19] and investigate countermeasures. As baseline target we used a Raspberry Pi 4, which is much less complex than contemporary smartphones. It is similar in complexity to the targets from [3], a ZTE ZFIVE and an Alcatel Ideal smartphone.

Measurement setup and target device We removed the metal lid to expose the die. Images of the chip and die can be found in Appendix C. The standard Raspbian OS was installed on the device. It was connected to Wi-Fi and we used SSH to interact with the device. Bluetooth was disabled. We used a LeCroy WavePro 804HD oscilloscope and various EM probes from Langer EMV.

5.1. Identification of activity-modulated signals

The work in [3] showed that sampling in a narrow band around the device’s clock frequency and processing the acquired signal with a sliding-median filter unveils sufficient information about the device’s activity, i.e., arithmetic operations and the conditional swap operations were visible. This was only partially reproducible with our setup. In particular for a clock frequency of 1.8 GHz, the CPU’s activity is hardly traceable and noise dominates the signal.

However, during our analyses, we noticed activity in frequency bands far below the CPU frequency which – nevertheless – seems clearly related to the CPU activity. Fig. 1 shows the frequency-filtered signals for OpenSSL + *secp521r1* and libcrypto + *Ed25519*. In particular, the absolute and sliding-median filtering of bandpass-filtered signals is helpful to detect repeating patterns. We used a window length of approximately $2\mu s$ for the sliding-median filter. For the *secp521r1* curve in OpenSSL, the peaks of this signal form a 5-2-1-2-3-1-3-3 pattern, which can be directly mapped to finite-field multiplication and square operations within the ladder step operation. The ladder step implementation uses the differential addition-and-doubling formulas from [51] (Appendix D). For libcrypto and *Ed25519*, the pattern cannot be directly mapped to operations within the double-and-always-add routine and the relevant frequency band is less dominant in the spectrum. We observed these effects for measurements at two locations. We placed a Langer EMV RF-B 2-3 probe on the die, above the location of the four CPUs (Fig. 11b), and a Langer EMV RF-U 2.5-2 probe on the center capacitor of the triplet marked with a blue rectangle in Fig. 11a. The signal on the capacitors looked more promising and the position is easier to reproduce, therefore we used it for all subsequent analyses. The capacitors act as a low-pass filter, which attenuate spectral components with higher frequencies such as the CPU frequency, but do not dampen the frequency bands we consider here.

The occurrence of distinct frequency bands for cryptographic operations was reported before, but its cause was

10. <https://github.com/openssl/openssl/pull/14464>

11. <https://github.com/openssl/openssl/pull/15702>

12. <https://github.com/openssl/openssl/pull/16543>

13. e.g., OpenSSL until release 1.1.1 (2018);

<https://openssl-library.org/news/openssl-1.1.1-notes/index.html>

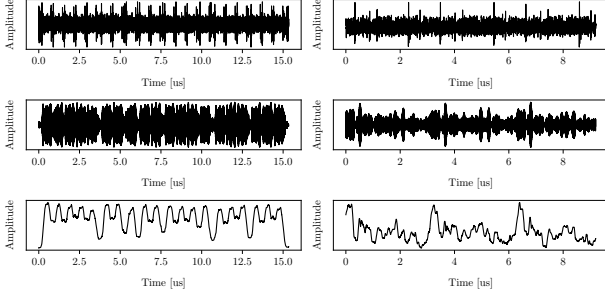


Figure 1: EM traces for a single Montgomery ladder step for OpenSSL + *secp521r1* (left) and three iterations for the double-and-always-add routine of libgcrypt and *Ed25519* (right). The top row shows the unfiltered signals. The traces in the middle row are bandpass-filtered for 130 MHz. The traces in the bottom row are additionally absolute and sliding-median filtered. The Raspberry Pi 4’s clock frequency was 1.8 GHz for this investigation.

never conclusively identified. Previous works have shown that distinct spectral components of operations can be used to break RSA and ECC implementations with secret-dependent branches [52, 5, 24, 6]. To trace the origin of these frequency bands, the authors of [6] recommended to explore the microarchitecture of processors. CPUs with an ARM Cortex-A8 architecture have two ALUs, only one of which contains a multiplier. The alternating use of these ALUs might explain the distinct pattern of finite-field multiply/square operations for OpenSSL and *secp521r1*. Other works – related to SCA aiming to recover user activity [53, 54, 55], covert channels [56], and malware detection [57] – observe that voltage regulator clocks and memory refreshes can also act as carriers for such activity-modulated signals. However, both carriers and activity-modulated signals are hard to identify and clearly associate with a hardware component even for dedicated micro-benchmarks [55]. Accordingly, it is difficult to assign the patterns in Fig. 1 to a carrier frequency with a corresponding hardware component. By artificially altering the CPU frequency, we could verify that the frequency of the modulated signal had a constant factor of roughly $1/14$ relative to the CPU frequency, e.g., if we manually enforce a clock frequency of 1.1 GHz, we observed the relevant activity around 80 MHz. This linear relationship suggests that a clock that scales linearly with the CPU frequency or the CPU frequency itself acts as carrier.

As long as the device is active (e.g., when a process like signature generation is triggered once in a while), it mostly uses a frequency of 1.8 GHz. The behavior described above would allow an attacker to easily adapt the attack to any CPU frequency.

5.2. Leakage assessment and exploitation

The conditional swap operation (Algorithm 2) occurs once before each step on the Montgomery ladder for OpenSSL or once after each iteration of the double-and-always-add routine in libgcrypt. As discussed in Section 4.2, the single-bit swap condition is amplified to multiple operations with distinct Hamming weights and distances. OpenSSL still implements the swap operation

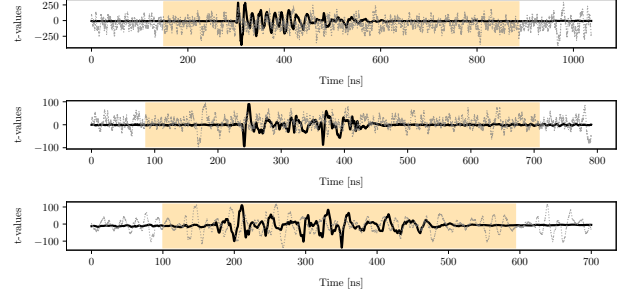


Figure 2: Leakage assessment results depicting t-values (solid, black line), a sample EM trace (dotted, grey line) and the trigger window (marked in orange). The top plot shows the results for OpenSSL + *secp521r1*, the middle plot for libgcrypt + *Ed25519* and the bottom plot for OpenSSL + *secp128r1* with the countermeasure from [3].

as described in Algorithm 2, whereas libgcrypt adopted the countermeasure described in Algorithm 3.

Algorithm 3 CT_SWAP with EM SCA countermeasure as implemented in libgcrypt.

Input: Two arrays a, b of size n machine words, representing a coordinate of an elliptic curve point; Swap condition $cond \in \{0, 1\}$.

Output:

If $cond = 1$: $a \leftarrow b, b \leftarrow a$;

If $cond = 0$: $a \leftarrow a, b \leftarrow b$.

- 1: $mask \leftarrow 0 - cond$ $\triangleright$ $mask$ is all-ones or zeros
 - 2: $\neg mask \leftarrow \neg(0 - cond)$ $\triangleright$ inverse $mask$
 - 3: **for** $i \leftarrow 0$ **to** $n - 1$ **do**
 - 4: $\delta_0 \leftarrow (a[i] \& \neg mask) \mid (b[i] \& mask)$
 - 5: $\delta_1 \leftarrow (a[i] \& mask) \mid (b[i] \& \neg mask)$
 - 6: $a[i] \leftarrow \delta_0$
 - 7: $b[i] \leftarrow \delta_1$
 - 8: **end for**
-

Algorithm 4 CT_SWAP with EM SCA countermeasure as proposed in [3].

Input: Two arrays a, b of size n machine words, representing a coordinate of an elliptic curve point; Swap condition $cond \in \{0, 1\}$; Random word r .

Output:

If $cond = 1$: $a \leftarrow b, b \leftarrow a$;

If $cond = 0$: $a \leftarrow a, b \leftarrow b$.

- 1: $mask \leftarrow 0 - cond$ $\triangleright$ $mask$ is all-ones or zeros
 - 2: **for** $i \leftarrow 0$ **to** $n - 1$ **do**
 - 3: $\delta \leftarrow (a[i] \oplus b[i]) \& mask$
 - 4: $\delta \leftarrow \delta \oplus r$
 - 5: $a[i] \leftarrow (a[i] \oplus \delta) \oplus r$
 - 6: $b[i] \leftarrow (b[i] \oplus \delta) \oplus r$
 - 7: **end for**
-

The authors of [3] were able to identify visually recognizable differences in the amplitude of the EM signal depending on the swap condition. Further, they relied on a k -means clustering algorithm with Euclidean distance metric to train a classifier capable of recovering the swap condition.

Leakage assessment We cannot directly classify traces visually with regard to their swap condition. Therefore, we relied on established SCA methodologies such as Welch’s t-test [58]. For this purpose, we recorded one million traces with a random, but known, swap condition. We used the Raspberry Pi 4’s GPIO pins to set a trigger signal at the beginning of the swap operation and reset it once it finishes. To keep jitter minimal, we allowed the program to run two step operations of the Montgomery ladder before we set the trigger for the single swap operation. If we isolate the swap operation, i.e., directly after the target program is started, the trigger pin is set and CT_SWAP is executed, the standard deviation of the time the trigger signal is active is twice as high. Otherwise, we followed the standard procedure for test vector leakage assessment (TVLA) [58], such as randomly selecting the swap condition for each trace to avoid microarchitectural effects. A detailed study regarding best-practices for TVLA on complex devices with dynamic frequencies and cache memories could be beneficial for future work. We used a sampling rate of 10 GHz and did not apply any filtering or alignment.

Fig. 2 shows the results of our leakage assessment. For all three of our experiments, we found a strong dependence of EM signals on the swap condition due to t-values far beyond the threshold of $|t| > 4.5$ [58]. This is obvious for OpenSSL + *secp521r1*, as no countermeasure is in place. The countermeasure libgcrypt adopted involves the inverse bit-mask $\neg mask$, which aims to eliminate certain parts of the leakage amplification, as both an all-ones and an all-zeros mask are used (Algorithm 3). However, since *mask* and $\neg mask$ are processed at different times, the leakage amplification still exists, even if a higher temporal resolution is needed to measure it. Further, the Hamming distance leakage of overwriting or not overwriting the arrays *a* and *b* in memory still exists. The authors of [3] proposed a different countermeasure described in Algorithm 4. It also eliminates part of the leakage amplification by introducing a random mask *r*. However, *mask* still amplifies the swap condition *cond* and so does the Hamming distance leakage of conditionally updating the arrays *a* and *b*. We implemented this countermeasure in OpenSSL and applied the same constraints as [3] to ensure that the compiler does not remove the countermeasure. We selected *secp128r1* as one of the curves with the least amount of leakage amplification. According to the t-test, there is still a strong dependence of EM emanations on the swap condition. Please note that the authors of [3] used a software-defined radio (SDR) with a lower sampling rate, lower bandwidth, and less ideal measurement positions for the attack and the evaluation of their countermeasure.

Evaluating classifiers We evaluated several methods to classify traces by swap condition. Neural networks, in particular, promise high accuracy and some immunity to temporal jitter [59, 60]. We trained classifiers on 800 000 traces of 1000 sample points, which were chosen according to the t-test. The remaining 200 000 traces were used for validation. For the evaluation of multiple classifiers, we used our measurements for OpenSSL + *secp521r1*. The highest accuracy we achieved is 98.14 % for a custom convolutional neural network (CNN) architecture. We derived the architecture by using a CNN pub-

lished as part of the ANSSI SCA database (ASCAD) [61] as basis for hyperparameter tuning. More details on the CNN architecture and the accuracy of other classifiers such as Gaussian templates can be found in Appendix E. For *Ed25519* in libgcrypt, the classifier attained a maximum accuracy of 98.03 %.

5.3. Attack evaluation

For the attack, we first needed to identify all swap operations. As discussed previously, we used activity-modulated signals to identify arithmetic operations. We automated this process and, with a few manual corrections, could identify all swap operations. For some traces we observed interruptions of the cryptographic operation, possibly related to actual interrupts, and interference signals, which do not interrupt the cryptographic operation, but overlay parts of the trace. The former can be counteracted with manual correction. The latter affects the probability with which the CNN can correctly classify the swap, but typically only up to ten swap operations are affected. For the classification of swap operations, we used the unfiltered signal, which was sampled at 2.5 GHz. Note that frequencies above 1.25 GHz experience aliasing for this sampling rate. This is inconsequential for our attack, because the activity-modulated signals of interest lie in lower frequency bands and the classification accuracy is hardly impacted, as evidenced by the results below.

We analyzed four traces per library. For OpenSSL and *secp521r1*, we correctly obtained 496, 508, 509 and 517 of 521 bits. For libgcrypt and *Ed25519*, we recovered 207, 210, 226, and 235 of 255 bits. While the countermeasure complicates the attack, a single-trace attack remains feasible, albeit with increased computational effort. Approaches to exploit multiple traces and signatures are discussed in Section 6.2.3.

6. Nonce@Once on a modern smartphone

In this section, we show two case studies, demonstrating the challenge when transferring the Nonce@Once attack to a modern smartphone target. First, we target the Fairphone 4 running a native Linux OS that allows root access. For the second case study, we use the Fairphone 4’s stock Android and embed the cryptographic library into a custom Android app. We focus only on OpenSSL and the *secp521r1* curve.

6.1. Case study 1: Fairphone 4 and native Linux

The Fairphone 4’s main SoC is the Snapdragon 750G 5G [62], which is also used in the Samsung Galaxy A52, Xiaomi Mi 10T, OnePlus Nord and other mid-range smartphones that were sold from 2021 and onward. As such, it is representative of smartphones that are slowly fading from the market, but are in active use for a few more years. As shown in Table 1, it is one of the most complex devices ever investigated for EM SCA to date. The goals of our first case study utilizing a Linux OS are to eliminate the complexity of the software stack as much as possible, to be able to apply

constraints to scheduling, and to find simple solutions for triggering. The Fairphone 4’s open ecosystem which provides support for alternative operating systems and an open PCB schematic¹⁴ facilitates our investigations. However, we anticipate that our results translate to other devices based on the same or similar SoCs.

Device preparation The Fairphone 4 is designed for repairability. Therefore, it was easy to remove the main PCB and disconnect unnecessary peripherals such as the camera. The metal lids covering the SoC and its peripherals on the opposite side of the PCB could be easily unclashed. We removed the plastic package of the SoC’s IC for some analyses, using an ULTRA TEC ASAP-1 CNC Mill. We emphasize that an attacker could also use a cheaper stationary drill. Except for the software, the laboratory setup was identical to the subsequent case study (Fig. 7). An infrared (IR) die-shot of the Snapdragon 750G 5G can be found in Appendix F. The phone was connected via USB and fully charged. For this case study, we unlocked the phone’s bootloader and installed postmarketOS¹⁵. This allows root access and limits the amount of background processes. The device’s Wi-Fi interface was disabled and no SIM card was installed.

Triggering To facilitate our analyses, we explored various ways of firing a trigger signal from the phone. We discovered that the I2C communication between the SoC and the AW8695 IC, which drives the phone’s vibration motor, provided an efficient method for triggering. The PCB schematic helped to identify the I2C bus’s pull-up resistors and solder a thin wire to them.

6.1.1. Investigation of CPUs

The Fairphone 4’s SoC incorporates two distinct CPU variants. The six Kryo 570 Silver / Cortex-A55 cores are optimized for energy efficiency, while the two Kryo 570 Gold / Cortex-A77 cores are optimized for performance. Using a PHEMOS-X emission microscope, we recorded an IR die-shot and the photon emission, when scheduling a program to a specific CPU. The results are shown in Fig. 3. The complete high resolution die-shot without the photon emission overlay can be found in Appendix F.

The attack must be tailored to the specific CPU executing the cryptographic operation. To investigate the electromagnetic profile of the CPUs, we placed a Langer EMV RF-B 3-2 probe over the respective area of the SoC and scheduled a scalar-by-point multiplication on the core.

The CPUs of the Snapdragon 750G 5G support a wide range of frequencies (Appendix F), with a maximal frequency of 1.8 GHz for A55 and 2.21 GHz for A77 CPUs and a minimal frequency of 300 MHz for both. When triggering the signature generation in intervals between one to three times per second, we observed that most of the time the A55 CPUs are clocked at 768 MHz and the A77 CPUs are clocked at 787 MHz while running the process. We consider this to be a realistic approximation of the actual attack, in which only one signature is generated, or the graphical user interface (GUI) limits the rate in which signatures can be created. We observed a shorter

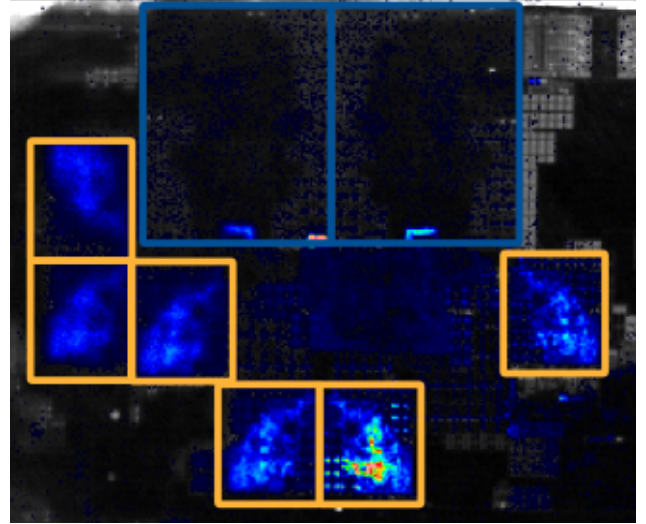


Figure 3: Excerpt of an IR die-shot of the Snapdragon 750G 5G SoC overlaid with photon emission measurements for CPU identification. In this case, a simple benchmark program is scheduled to one of the six A55 CPUs marked with orange rectangles. The two A77 CPUs are highlighted with blue rectangles.

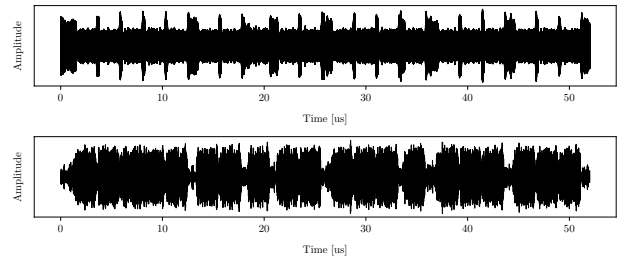


Figure 4: Single step on the Montgomery ladder on one of the Snapdragon 750G 5G’s Cortex-A55 CPUs bandpass-filtered for 768 MHz (top) and 40 MHz (bottom).

run time due to a higher core frequency in only about one in ten traces. In contrast to the Raspberry Pi 4, the Snapdragon 750G 5G seems to scale its CPU frequency more conservatively, most likely due to energy efficiency.

Fig. 4 and Fig. 5 contain bandpass-filtered signals for a single ladder step on the Montgomery ladder if scheduled on an A55 or A77 CPU with the core frequencies described above. For the A55 core (Fig. 4), we observed a similar modulation effect as for the Raspberry Pi 4 at frequencies of $f_{mod_l} = 41$ MHz and $f_{mod_h} = 82$ MHz. Applying a 41 MHz bandpass filter unveils the distinctive pattern of finite-field multiplication and square operations we also observed on the Raspberry Pi 4 (Fig. 1). When

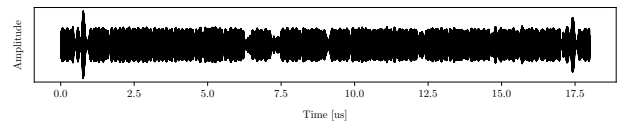


Figure 5: Single step on the Montgomery ladder on one of the Snapdragon 750G 5G’s Cortex-A77 CPUs bandpass-filtered for 787 MHz.

14. https://www.fairphone.com/wp-content/uploads/2022/09/FP4_Information-for-repairers-and-recyclers.pdf

15. <https://postmarketos.org/>

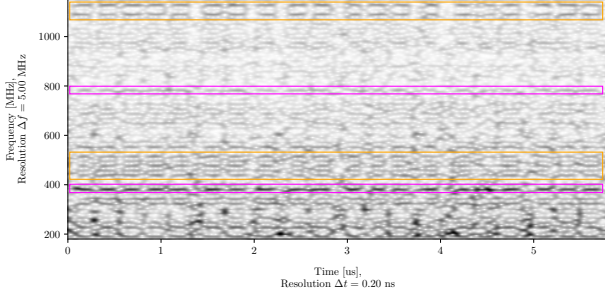


Figure 6: STFT for a single step on the Montgomery ladder on one of the Snapdragon 750G 5G’s Cortex-A77 CPUs with a frequency of 2.21 GHz. The distinct pattern of repeating blocks in a 5-2-1-2-3-1-3-3 structure can be observed on multiple frequency bands.

artificially forcing higher clock frequencies, we observed that $f_{mod_h} = 2f_{mod_l}$ always holds and – as observed on the Raspberry Pi – the factors $f_{mod_l/h}/f_{cpu}$ remain constant. Since we were measuring on the die and not – as for the Raspberry Pi 4 – on a capacitor, which attenuates higher frequencies, we could also extract information from a signal that has been filtered for the CPU frequency of 768 MHz. In this case, the amplitude is increased for the time intervals between finite-field multiplications and square operations.

For the A77 CPU (Fig. 5) and a CPU frequency of 787 MHz, we get the best signal by applying a bandpass filter around the CPU frequency. Fig. 6 shows the Short-Time Fourier Transformation (STFT) result when forcing the CPU frequency to the maximum of 2.21 GHz. In this case, filtering for the CPU frequency leads to noisy traces, from which operations are not directly apparent. However, we detected two kinds of useful activity-modulated signals. Symmetric to $1/2f_{cpu} = 1.105$ GHz and $1/4f_{cpu} = 552.5$ MHz, we observed the modulated activity in two, respectively four bands (marked in orange in Fig. 6). This seems to indicate that the CPU frequency serves as the carrier and that the cryptographic activity is modulated onto various harmonics, which scatter for higher-order harmonics. Additionally, we observed activity-modulated signals at 390 MHz and 790 MHz (marked in magenta in Fig. 6). By varying the CPU’s frequency, we confirmed that the factors f_{mod}/f_{cpu} were also constant here.

We were thus able to confirm that helpful activity-modulated signals exist at frequencies well below the CPU frequency for both CPU variants present in the Snapdragon 750G 5G. This is particularly important for high CPU frequencies, where we observe significant attenuation by the package when measuring with and without the package. Our results confirm that the carrier is most likely the CPU’s clock frequency. For some spectral components (e.g., the ones marked in orange in Fig. 6), the factor f_{mod}/f_{cpu} is easy to explain, whereas for others (e.g., the ones marked in magenta in Fig. 6), we can only assume that the cause lies in the microarchitecture of the CPU combined with periodicity in the running software.

6.1.2. Results for Nonce@Once attack

To reduce the complexity of our first case study, we constrained the scheduling via *taskset* such that a fixed A55 CPU was used for the scalar-by-point multiplication. This simplified the attack significantly, as we could place the probe accordingly. The increased complexity of a mobile platform and the more advanced semiconductor node are the most impactful challenges in this case study. We used the device for which we removed the package. As in Section 5.3, we used a sampling rate of 2.5 GHz.

Although our I2C triggering mechanism works reliably, its repetition rate is several orders of magnitude lower than that of a conventional GPIO trigger and it introduces more jitter. To acquire a sufficient amount of reliable profiling data, we slightly modified the Montgomery ladder code we ran on the device during the profiling phase to extract as many swap operations as possible per trace and label them correctly. In short, we used only nonces that correspond to always or never swapping and random base points for the scalar-by-point multiplication. We acquired two million training traces in less than 24 h. The trained CNN achieved a final accuracy of 99.8% during validation.

For the attack phase, we ran the unmodified OpenSSL code. We used the artificial trigger signal for convenience. Our results show that an amplitude-based trigger on a bandpass-filtered signal should also be sufficient. The alignment of swap operations worked similar to the one described in Section 5.3, using the activity-modulated signal around 41 MHz and similar absolute, sliding-median filters. Overall, we identified slightly more interruptions and interference signals than on the Raspberry Pi 4. We evaluated the attack on two traces, where we successfully recovered 505 and 517 bits of the respective nonces.

Even though the initial investigation required more effort, the attack in this scenario is ultimately just as easy to carry out as on the Raspberry Pi 4. CPU pinning and a favorable measurement position on the die make the attack easier. The A55 core’s comparatively low operating frequency also helps. However, this is not due to artificial constraints, but reflects normal dynamic frequency scaling, where the CPUs increase frequency only under higher load.

6.2. Case study 2: Fairphone 4 and Android

In this case study, we examine how the full Android software stack affects the feasibility of SCA. Our target is a simple, custom Android app that includes native OpenSSL code. Please note that in general, developers are recommended to use the Android Keystore. We discussed our motivation for investigating libraries such as OpenSSL in Section 4.2.

Setup The setup is shown in Fig. 7. The hardware was the same as in the previous case study. We flipped the phone’s display to place a probe on the SoC’s die. For convenience, we again used an artificial trigger signal, the necessity of which will be discussed later. We could not use the I2C bus from the first case study, since Android uses it independently for other functions. Instead, we relied on the pulse width modulation (PWM) signal that drives the phone’s vibration motor, as it is simple

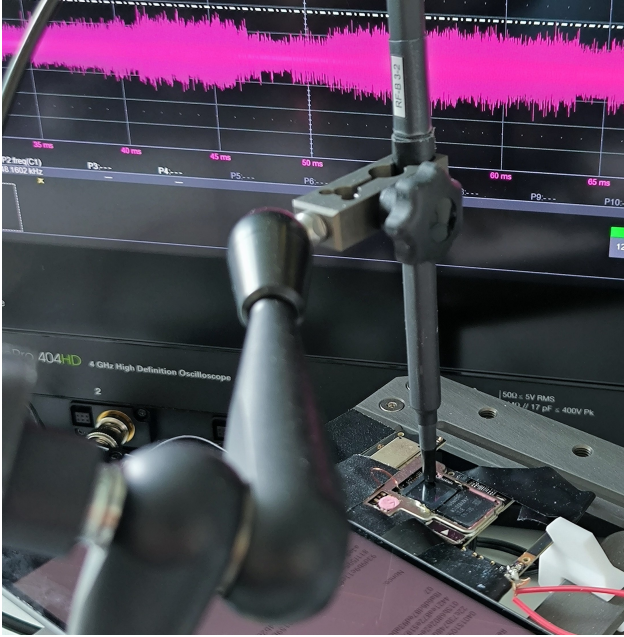


Figure 7: Measurement setup for EM SCA of the Fairphone 4 app target.

to trigger a vibration from our target app. The phone was running an unmodified Android 13. Wi-Fi was not connected but enabled, no SIM card was installed. The phone is connected to a host PC via USB and charging. We used ADB’s *uiautomator* to interact with the target app.

6.2.1. Scheduling investigation

Without the scheduling constraints of the previous case study, the first challenge is to understand where the cryptographic operation is scheduled. First, we analyzed the photon emission, which we also used to locate individual CPUs, in order to obtain information about the scheduling behavior. During this analysis, the phone was fully charged and connected to power. Our main findings are that the device utilizes one of the A77 CPUs for Android’s background tasks. Individual native processes with computation times below 1s are executed on the same core. It is unclear why the system utilizes a performance core instead of an energy-optimized CPU for this minimal load, but the full-charge status and connection to a power supply might be the cause for this. For longer processes, we observed that an additional core, mostly one of the A55 CPUs, was used. Further, we investigated the impact of Android apps, the load introduced by handling touch events, and the scheduling of our target app with native OpenSSL code. We observed that an Android app, even if idle, caused increased activity on three A55 and one A77 CPUs. Handling touch events further increased the activity on A55 CPUs, however we cannot make any statements about where cryptographic operations triggered by a touch event are scheduled.

We conducted EM measurements to gain further insights. Although the 2 mm resolution of the Langer EMV RF-B 3-2 probe is too fine to cover the complete area occupied by CPUs (Fig. 3), we found a position where we can measure the EM emanations for all eight CPUs.

The use of Android and an app significantly increases the noise in the traces. Fig. 8 shows that the operation is impossible to detect without a filter and is also potentially overlaid by noise signals. However, we could recognize the characteristic activity-modulated signals we established in Section 6.1. We automated the detection of the relevant patterns in the characteristic frequency ranges and applied it to 100 traces. For 57, we could identify the Montgomery ladder on an A77 CPU clocked at 787 MHz. For the remainder of traces, the Montgomery ladder was either scheduled on an A55 CPU, the A77 core was clocked at a different frequency, or the trace was too noisy for automatic detection.

This makes the A77 CPU with a frequency of 787 MHz the most likely scheduling scenario. At least, this holds for scenarios similar to our laboratory setup, i.e., the phone is fully charged and connected to power. We also suspect that the app design has an impact, e.g., if it causes a higher load due to other activities, the clock frequency might be higher.

6.2.2. Results for Nonce@Once attack

We decided to tailor the attack to the A77 CPU and a core frequency of 787 MHz in this case study. This means that we ignored traces where the A77 CPU used a different frequency or the cryptographic operation was scheduled on an A55 CPU. We identified such traces from their characteristic spectral components and the duration of the operation. As we showed above, scheduling on an A77 core with a frequency of 787 MHz is the most likely scenario for our setup and target app. For an actual attack, an adversary should either prepare classifiers for all CPU variants and their most likely frequencies or be certain that enough traces can be recorded such that at least for one trace, the cryptographic operation is scheduled on the desired CPU with the desired frequency. However, the number of necessary traces is not only relevant for different scheduling scenarios, but also depends on the number of bits that can be recovered per trace. We discuss this in more detail below.

Training trace acquisition and classifier training The easiest method for acquiring training traces would be to again use postmarketOS. This would allow us to enforce the scheduling on the desired CPU, enforce the frequency, and reduce the measurement time significantly, as the I2C-based trigger could be used. However, this might not be possible for every phone, therefore we explored the effort needed to acquire enough training traces on a phone running Android and our target app. The advantage of this is that effects such as random use of one of the two A77 CPUs and increased noise due to more background activity are covered by the training data in a natural way. The usage of the vibration motor as trigger signal limits the amount of measurements to a single trace per second. Further, as discussed above, we discarded traces with non-matching scheduling conditions. For training trace acquisition, we re-used the modifications to the Montgomery ladder described in Section 6.1 and obtained around two million training traces in five days of measurement time. As in all previous attacks, we used a sampling rate of 2.5 GHz. The trained CNN achieved a validation accuracy of 92 %. This is significantly lower than in the previous

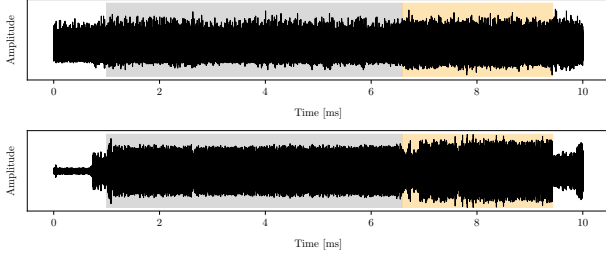


Figure 8: Trace of the scalar-by-point multiplication within an Android app without (top) and with (bottom) bandpass-filter for 787 MHz. For the sub-trace highlighted in grey, swap alignment and recovery works with a high probability. An interference signal of unknown origin prevents this for the sub-trace highlighted in orange.

case study. Possible explanations for this are increased noise, the complexity of the A77 CPU, and the variable scheduling between the two A77 CPUs.

Attack phase For the attack phase, we recorded 20 traces. We used our artificial vibration trigger. Triggering on a bandpass-filtered EM signal should be possible; alternatively, pattern-based triggering could be explored. Of the 20 traces, 11 seem to be applicable to our classifier, i.e., an A77 CPU clocked at 787 MHz is used. Most of these traces are affected by intermittent interference signals as shown in Fig. 8. This increases the manual effort required to align swap operations. We analyzed two traces for which we recovered 416 and 387 of the 521 bits correctly. Both traces include interference signals as depicted in Fig. 8. We cannot definitively identify the cause of the interference. It could be related to a frequency scaling or activity from another CPU. Within the superimposed part of the trace, the success rate is equivalent to randomly guessing the bit. For the remainder of the traces, where swap operations are aligned with a high confidence, the CNN almost achieves the same accuracy as during validation. With $521 - 416 = 105$ or more incorrect/missing bits, obtaining the complete nonce in order to derive the secret key from the signature is no longer trivial.

6.2.3. Discussion of key recovery methods

We assume that 416 and 387 bits per trace are not tight upper and lower bounds, respectively, on the number of bits that can be recovered. Since most of the 11 traces show similar interference patterns, we used these values as the basis for analyzing key recovery methods. With the given classifier and its validation accuracy, we expect approximately $521 \cdot 0.08 \approx 42$ bits to be erroneous, even if a trace is not overlaid by any interference signal. First, we analyze to what extent the CNN’s score/probability values for each guess can be used as an indication of potentially wrong nonce bits. Knowing the indices of these bits is the difference between guessing 42 bits or trying $\binom{521}{42} \approx 2^{206.7}$ combinations for an interference-free trace.

Fig. 9 shows that the CNN’s average probability value is around 0.9 for correct and 0.75 for incorrect guesses. Therefore, many incorrect bits can be identified from the CNN’s confidence, but the standard deviation of the probabilities for incorrect guesses shows that not all in-

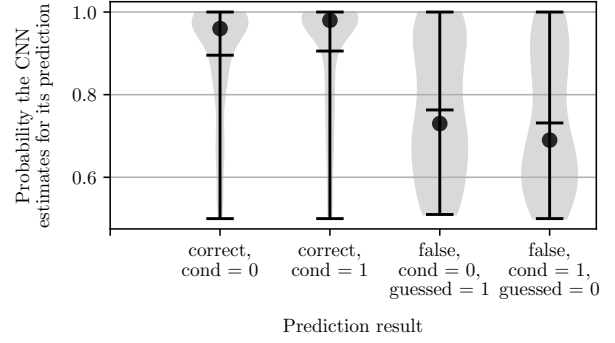


Figure 9: Violin plot [63] of the distribution of probabilities the CNN estimates for its predictions. The vertical line highlights the distribution’s mean, the circle marks the distribution’s median.

correct bits can be identified reliably. This increases the brute-force effort beyond 42 bits. Thus, even a completely interference-free trace is not guaranteed to enable a trivial brute-force recovery of the secret key.

If we assume that – due to interference – at most 416 bits per trace can be recovered, it is obvious that brute-force recovery is not feasible. In this case, multiple traces, where each trace corresponds to a dedicated nonce - signature pair, can be utilized for the attack. From the partial information about the nonce k for each respective trace and the signature equation (Section 2.2), a hidden number problem (HNP) [64] can be formulated, where the secret key d_A corresponds to the hidden number. This HNP instance can be solved either by Fourier analysis [65, 66, 67] or lattice-based recombination [68, 69, 70, 71]. In the following, we conduct a short study based on recent work on lattice-based recombination [71]. In general, approaches based on Fourier analysis are believed to enable key recovery in scenarios where the number of leaking bits is too low for lattice-based recombination. However, recent work claimed to close this gap for lattice-based recombination [71]. Further, lattice-based recombination usually requires significantly fewer signatures, which – considering our threat models – is an important metric.

Lattice-based recombination transforms the HNP into a unique shortest vector problem (uSVP). The way the lattice is constructed and then solved for the target vector was discussed in multiple works [68, 69, 70, 71]. The latest results [71] improved the lattice construction and algorithms originally proposed in [68] and covered a wide range of possible attack scenarios.

The challenge in applying their work to our scenario lies in the handling of errors. The error rate in [71] was defined for each equation related to the HNP and used to construct the lattice. This means the probability of error p_e is related to the probability that at least one of the l recovered nonce bits for one nonce - signature pair is incorrect. The success rate we used so far in this paper is per-bit, i.e., p_b is the probability that a single bit is correct. Since we recover each bit individually, $p_e = 1 - (p_b)^l$. It is obvious that, even for significantly higher p_b than we currently achieve, for large l , p_e will always be relatively high, e.g., for $p_b = 0.99$ and $l = 100$, $p_e = 0.63$.

An additional constraint is that solvers from literature

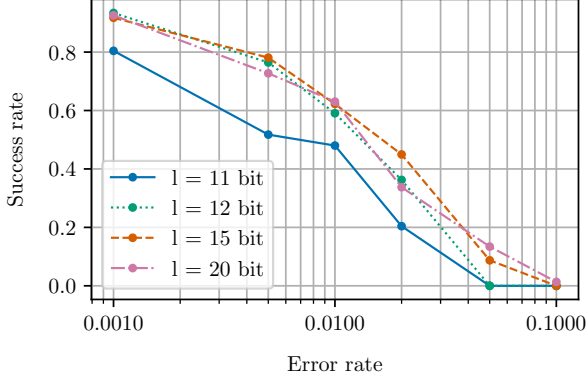


Figure 10: Success rate over error rate p_e for key recovery by lattice recombination using the solver from [71]. We used 50 nonce - signature pairs (with $x = 1$) where l bits of the nonce are known. The results are for the *secp521r1* curve. For each parameter set, the success rate was calculated for 100 trials.

assume the l bits to be in a consecutive chunk starting from the nonce’s most- or least-significant bit.

We used the solver from [71] to evaluate which error rates can be tolerated for $l \leq 20$, using 50 nonce - signature pairs¹⁶. The results are shown in Fig. 10. We can see that, for a good chance at key recovery, $p_e \leq 0.01$ is required. For $l = 20$ this means $p_b > 0.9995$, and for $l = 11$, $p_b > 0.999$ is needed.

From the data used for Fig. 9, we evaluated the feasibility of pruning, i.e., not considering bits for which the CNN’s confidence is below a certain threshold. For example, if we only consider bits for which the CNN’s probability is 0.99 or higher, we get $p_b = 0.946$. In this case, we must abandon 65.7 % of bits during pruning.

This shows that the success rates we achieved in our laboratory setup are not sufficient for key recovery with the parameters used for Fig. 10. For $l \leq 10$, this might change, but such parameter choices make the computation highly time-consuming. For reference, we cite a result from [70], who estimate that for a 521-bit modulus and $p_e = 0$, at least 66 and 132 signatures are needed for 8- and 4-bit leakage, respectively.

We conclude that a solver suitable to our scenario is currently missing in literature, i.e., a solver that can tolerate higher error rates and is more flexible with regard to the position of the leaking bits. Our current experimental results are not directly applicable to existing solvers, but serve as a benchmark for the accuracy a measurement setup must achieve.

7. Impact on smartphone security

In this section, we broaden our focus and discuss the implications of our results for smartphone security.

¹⁶ We use $x = 1$, which makes the lattice equivalent to the one proposed in [68]. While greater values for x could be beneficial for our scenario, they massively increase the number of required signatures to a point where our specific attack would clearly be infeasible.

7.1. Limitations

Our evaluation has several practical limitations that affect the generality of the results.

Invasiveness. The attack relies on invasive measurements requiring physical modification of the device. In [3], the Nonce@Once attack was demonstrated on older smartphones, where an EM probe placed on the device’s casing was sufficient. This would allow for scenarios in which the victim remains in possession of the device and does not notice the attack (Section 3.2). Therefore, we examined less invasive probe placements for the Fairphone 4. For both measurements on the IC and decoupling capacitors, we observed that the metal lids on top of them block the signal such that spectral components we use for alignment are almost completely attenuated. We did not observe changes in the amplitude of measurements when starting a process. The touchscreen above the IC and the casing on top of the capacitors eliminated the spectral components completely. While more advanced probes or signal processing techniques may improve this, demonstrating a fully non-invasive attack remains an open problem.

Apart from the probe position it is also interesting to note that the authors of [3] used a relatively inexpensive SDR with a bandwidth of only 40 MHz. In comparison, we use more sophisticated laboratory equipment. However, compared to the probe placement, this has less influence on the threat posed by EM SCA to smartphones. At this point, we also emphasize that we used an artificial trigger for all our analyses. This should be relatively easy to replace with pattern-based triggering on bandpass-filtered signals.

Transferability. Our experiments use a template attack with profiling and attack traces collected on the same device. Cross-device transferability would likely require more robust feature extraction and training across multiple devices, due to manufacturing differences and inaccurate probe placements. Wrt. transferability to other elliptic curves, our analysis in Section 5 indicated that the attack should also work for curves with lower leakage amplification.

Generality. We evaluated a single mid-range smartphone platform. While the underlying leakage mechanisms are expected to persist across similar architectures, modern smartphones increasingly employ heterogeneous multi-core designs, higher clock frequencies, and smaller technology nodes. This complicates drawing general conclusions about whether our findings are applicable to other devices with different chipsets.

Another potentially influential factor is package-on-package (PoP) architectures, in which memory is mounted on top of the IC. These are becoming increasingly common in smartphones and could make attacks much more difficult, as local measurements on the IC are no longer possible without additional physical preparation [7, 8, 9].

Regarding the transferability to a concrete app implementation, we want to emphasize that a multitude of operational conditions play a critical role. Depending on the app architecture, the smartphone’s battery level, and background activity – including that of the app itself and

the OS – clock frequency and scheduling may vary more significantly. For newer smartphones, which sometimes feature up to three different CPU variants and support a broader range of frequencies, even exceeding 3 GHz, this results in a vast evaluation space. However, for an attacker with high attack potential as it is for example assumed in [72], this additional effort is expected to have no significant impact on the attack’s assessment. Our investigations in Section 6.1 showed that activity-modulated signals in lower frequency bands can help implement the attack for higher clock frequencies such as 2.21 GHz.

Finally, it should be noted that Android applications should use the Android Keystore, which – for most devices – uses a proprietary library within a TEE or Strong-Box. In this case, the feasibility of attacks depends heavily on the implementation within this library and whether it includes countermeasures.

Complete key recovery Our analysis in Section 6.2.3 highlighted that a complete key recovery is not trivial for measurements of an Android app. The investigation of solvers that can efficiently handle noisy leakage and the improvement of this leakage remain open challenges.

7.2. Threat assessment

The applicability of our investigated attacks strongly depends on the threat model. For the scenario of an unlocked smartphone under adversarial control (Section 3.1), our case studies directly apply. In this setting, an attacker can trigger cryptographic operations and potentially recover ECDSA private keys, enabling unauthorized signing. As discussed in Section 3.1, stolen, but locked devices may fall into this category, if the lock mechanism can be bypassed. If it cannot be bypassed, the attack surface is reduced to cryptographic operations which the locked phone might conduct automatically (Section 3.2). This highlights that strong locking mechanisms, which are also used to gate access to cryptographic operations, are an effective first line of defense.

For scenarios in which the device remains in possession of the user (Section 3.2), the attack is currently not practical. Our measurements rely on invasive probe placements that require disassembling the device and removing shielding components. This excludes realistic attack scenarios in which the adversary has no prolonged physical access to the device. Here we assume that the attacker is not the owner of the device (Section 3.1).

Overall, our results demonstrate that hardware side-channel attacks are difficult to mitigate purely in software. For critical identity solutions such as the EUDI wallet, short-term mitigation includes algorithmic countermeasures and protocol-level protections such as requiring a second factor (e.g., an eID card).

7.3. Countermeasures

In the course of our analysis of the attack methodology, we found that the countermeasure libgcrypt adopted does not provide sufficient protection – at least in the context of our laboratory setup and the relatively simple Raspberry Pi 4 platform. Our leakage assessment

results suggest that this also applies to the countermeasure described in [3] (Section 5.2). Consequently, the question of effective mitigation remains unresolved. As described in Section 4.2, widespread countermeasures such as coordinate re-randomization, point blinding, and scalar blinding are not or only partially effective against the attack. Swapping pointers instead of the actual coordinates could reduce leakage amplification, but would introduce secret-dependent memory access patterns [3]. The authors of [73] propose to combine scalar blinding, address randomization [74, 75], and projective coordinate re-randomization [76]. Scalar blinding implies that an adversary must recover all swap operations with a high accuracy, since only then it is possible to recover the actual scalar from the blinded one. Further, address randomization implies that two swap conditions must be recovered successfully per bit. The leakage amplification for both swap conditions can be reduced, if projective coordinate re-randomization, and the countermeasure from [3] are combined carefully. That is, each coordinate must be completely re-randomized before it is written back, ensuring that values in memory are always updated. The practical validation of countermeasures is left open as future work.

8. Conclusion

In this work, we conduct an analysis of the practicality of EM SCA on modern smartphones, utilizing the Fairphone 4’s Snapdragon 750G 5G SoC as an exemplary target platform. Our main contributions include the adaptation and execution of conditional-swap SCA on ECC [3, 18, 19] against recent hardware and software stacks, demonstrating that such attacks remain a tangible threat to contemporary smartphones. Our case studies demonstrate that challenges such as heterogeneous CPU clusters, dynamic frequency scaling, scheduling, and parallel activities of the device can be overcome. We further show that the countermeasures proposed by [3] and implemented in libgcrypt are insufficient in practice. Notably, this is the first work to systematically investigate a device as complex as the Fairphone 4 for EM SCA, analyzing full app contexts rather than isolated cryptographic libraries. Beyond our practical investigations, we provide an overview of how cryptography is implemented on Android smartphones and where potential vulnerabilities lie. We define two realistic threat models that guided our evaluation and allowed us to assess the real-world impact of our attack approaches.

Looking ahead, our findings make clear that understanding the full attack surface of smartphones is only just beginning. Many open questions remain, in particular regarding the feasibility of attacks that do not require the smartphone to be disassembled and other limitations listed in Section 7. Our work serves as an important step towards a deeper understanding of real-world risks, and we hope it motivates the community to further explore side-channel security on complex mobile platforms.

Finally, we see our work in the context of critical identity solutions akin to the EUDI wallet. Our results show clearly that such applications must be implemented with great care, and that in the long run, all smartphones need to be equipped with certified, secure hardware in which users can place their trust.

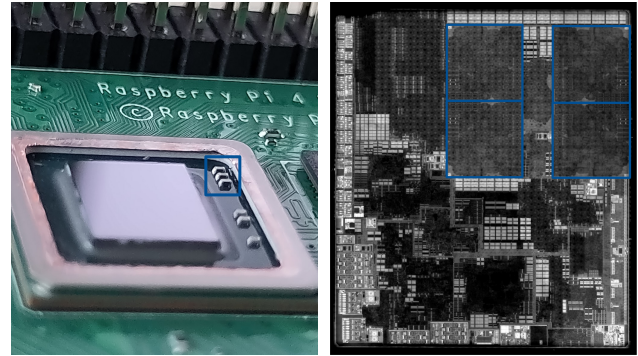
References

- [1] Council of the European Union, “Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework,” Online, 2024. [Online]. Available: <http://data.europa.eu/eli/reg/2024/1183/oj>
- [2] I. Kuznetsov, V. Pashkov, L. Bezvershenko, and G. Kucherin, “Operation Triangulation: iOS devices targeted with previously unknown malware,” Online, 2023. [Online]. Available: <https://securelist.com/operation-triangulation/109842>
- [3] M. Alam, B. B. Yilmaz, F. Werner, N. Samwel, A. G. Zajic, D. Genkin, Y. Yarom, and M. Prvulovic, “Nonce@Once: A single-trace EM side channel attack on several constant-time elliptic curve implementations in mobile platforms,” in *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2021, pp. 507–522. [Online]. Available: <https://doi.org/10.1109/EuroSP51992.2021.00041>
- [4] M. Alam, H. A. Khan, M. Dey, N. Sinha, R. L. Callan, A. G. Zajic, and M. Prvulovic, “One&done: A single-decryption EM-based attack on OpenSSL’s constant-time blinded RSA,” in *27th USENIX Security Symposium (USENIX Security 2018)*. USENIX Association, 2018, pp. 585–602. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/alam>
- [5] D. Genkin, L. Pachmanov, I. Pipman, E. Tromer, and Y. Yarom, “ECDSA key extraction from mobile devices via nonintrusive physical side channels,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS) 2016*. ACM, 2016, pp. 1626–1638. [Online]. Available: <https://doi.org/10.1145/2976749.2978353>
- [6] P. Belgarric, P.-A. Fouque, G. Macario-Rat, and M. Tibouchi, “Side-Channel Analysis of Weierstrass and Koblitz Curve ECDSA on Android Smartphones,” in *Topics in Cryptology - CT-RSA 2016*. Springer, 2016, pp. 236–252. [Online]. Available: https://doi.org/10.1007/978-3-319-29485-8_14
- [7] O. Lisovets, D. Knichel, T. Moos, and A. Moradi, “Let’s Take it Offline: Boosting Brute-Force Attacks on iPhone’s User Authentication through SCA,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 3, pp. 496–519, 2021. [Online]. Available: <https://doi.org/10.46586/tches.v2021.i3.496-519>
- [8] A. Vasselle, P. Maurine, and M. Cozzi, “Breaking Mobile Firmware Encryption through Near-Field Side-Channel Analysis,” in *ASHES@CCS 2019*. ACM, 2019, pp. 23–32. [Online]. Available: <https://doi.org/10.1145/3338508.3359571>
- [9] G. Haas and A. Aysu, “Apple vs. EMA: electromagnetic side channel attacks on Apple CoreCrypto,” in *59th ACM/IEEE Design Automation Conference (DAC)*. Association for Computing Machinery, 2022, pp. 247–252. [Online]. Available: <https://doi.org/10.1145/3489517.3530437>
- [10] S. Bhasin, H. Boyapally, and D. Jap, “Reality Check on Side-Channels: Lessons learnt from breaking AES on an ARM Cortex A processor,” *Cryptology ePrint Archive*, Paper 2024/1381, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1381>
- [11] J. Longo, E. D. Mulder, D. Page, and M. Tunstall, “SoC It to EM: ElectroMagnetic Side-Channel Attacks on a Complex System-on-Chip,” in *Cryptographic Hardware and Embedded Systems (CHES) 2015*, ser. Lecture Notes in Computer Science, vol. 9293. Springer, 2015, pp. 620–640. [Online]. Available: https://doi.org/10.1007/978-3-662-48324-4_31
- [12] G. Goller and G. Sigl, “Side Channel Attacks on Smartphones and Embedded Devices Using Standard Radio Equipment,” in *COSADE 2015*, ser. Lecture Notes in Computer Science, vol. 9064. Springer, 2015, pp. 255–270. [Online]. Available: https://doi.org/10.1007/978-3-319-21476-4_17
- [13] C. Shepherd, K. Markantonakis, N. van Heijningen, D. Aboukassimi, C. Gaine, T. Heckmann, and D. Naccache, “Physical fault injection and side-channel attacks on mobile devices: A comprehensive analysis,” *Computers & Security*, vol. 111, p. 102471, 2021. [Online]. Available: <https://doi.org/10.1016/j.cose.2021.102471>
- [14] P. Cronin, X. Gao, C. Yang, and H. Wang, “Charger-Surfing: Exploiting a Power Line Side-Channel for Smartphone Information Leakage,” in *30th USENIX Security Symposium (USENIX Security 2021)*. USENIX Association, 2021, pp. 681–698. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/cronin>
- [15] A. P. Sayakkara and N.-A. Le-Khac, “Forensic Insights From Smartphones Through Electromagnetic Side-Channel Analysis,” *IEEE Access*, vol. 9, pp. 13 237–13 247, 2021. [Online]. Available: <https://doi.org/10.1109/ACCESS.2021.3051921>
- [16] L. Navanesan, K. de Zoysa, and A. P. Sayakkara, “Impact of Multiple CPU Cores to the Forensic Insights Acquisition From Mobile Devices Using Electromagnetic Side-Channel Analysis,” *IEEE Access*, vol. 13, pp. 94 953–94 969, 2025.
- [17] A. Wang, P. Gopalkrishnan, Y. Wang, C. W. Fletcher, H. Shacham, D. Kohlbrenner, and R. Paccagnella, “Pixnapping: Bringing pixel stealing out of the stone age,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2025.
- [18] E. Nascimento, Ł. Chmielewski, D. Oswald, and P. Schwabe, “Attacking Embedded ECC Implementations Through cmov Side Channels,” in *Selected Areas in Cryptography – SAC 2016*. Springer, 2017, pp. 99–119.
- [19] E. Nascimento and Ł. Chmielewski, “Applying Horizontal Clustering Side-Channel Attacks on Embedded ECC Implementations,” in *Smart Card Research and Advanced Applications*. Springer, 2018, pp. 213–231.
- [20] P. C. Kocher, “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems,” in *Advances in Cryptology — CRYPTO ’96*. Springer, 1996, pp. 104–113.
- [21] P. Kocher, J. Jaffe, B. Jun, and P. Rohatgi, “Introduction to Differential Power Analysis,” *Journal of Cryptographic Engineering*, vol. 1, no. 1, pp. 5–27, 2011. [Online]. Available:

- <https://doi.org/10.1007/s13389-011-0006-y>
- [22] D. Agrawal, B. Archambeault, J. R. Rao, and P. Rohatgi, “The EM Side—Channel(s),” in *CHES 2002*. Springer, 2003, pp. 29–45.
 - [23] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, “Spectre Attacks: Exploiting Speculative Execution,” in *IEEE Symposium on Security and Privacy (SP) 2019*, 2019, pp. 1–19.
 - [24] D. Genkin, L. Pachmanov, I. Pipman, and E. Tromer, “ECDH key-extraction via low-bandwidth electromagnetic attacks on PCs,” in *Topics in Cryptology - CT-RSA 2016*. Springer, 2016, pp. 219–235.
 - [25] P. Markert, D. V. Bailey, M. Golla, M. Dürmuth, and A. J. Aviv, “On the Security of Smartphone Unlock PINs,” *ACM Transactions on Privacy and Security*, vol. 24, no. 4, pp. 1–36, 2021. [Online]. Available: <https://doi.org/10.1145/3473040>
 - [26] S. Uellenbeck, M. Dürmuth, C. Wolf, and T. Holz, “Quantifying the Security of Graphical Passwords: The Case of Android Unlock Patterns,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS) 2013*. Association for Computing Machinery, 2013, pp. 161–172. [Online]. Available: <https://doi.org/10.1145/2508859.2516700>
 - [27] A. J. Aviv, K. Gibson, E. Mossop, M. Blaze, and J. M. Smith, “Smudge attacks on smartphone touch screens,” in *USENIX Workshop on Offensive Technologies (WOOT) 2010*. USENIX Association, 2010.
 - [28] Y. Chen, Y. Yu, and L. Zhai, “InfinityGauntlet: brute-force attack on smartphone fingerprint authentication,” in *32nd USENIX Security Symposium (USENIX Security '23)*. USENIX Association, 2023.
 - [29] I. Goicoechea-Telleria, R. Sanchez-Reillo, J. Liu-Jimenez, and R. Blanco-Gonzalo, “Attack Potential Evaluation in Desktop and Smartphone Fingerprint Sensors: Can They Be Attacked by Anyone?” *Wireless Communications and Mobile Computing*, vol. 2018, no. 1, p. 5609195, 2018. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2018/5609195>
 - [30] T. Ni, X. Zhang, and Q. Zhao, “Recovering Fingerprints from In-Display Fingerprint Sensors via Electromagnetic Side Channel,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS) 2023*. Association for Computing Machinery, 2023, pp. 253–267. [Online]. Available: <https://doi.org/10.1145/3576915.3623153>
 - [31] Z. Zheng, Q. Wang, and C. Wang, “Spoofing Attacks and Anti-Spoofing Methods for Face Authentication Over Smartphones,” *IEEE Communications Magazine*, vol. 61, no. 12, pp. 213–219, 2023.
 - [32] Google, “Google System Services Release Notes,” Online, 2025, accessed: 2025-09-16. [Online]. Available: <https://support.google.com/product-documentation/answer/14343500>
 - [33] Android Developers, “Android Keystore System,” Online, 2025, accessed: 2025-09-16. [Online]. Available: <https://developer.android.com/privacy-and-security/keystore>
 - [34] E. Leierzopf, R. Mayrhofer, M. Roland, W. Studier, L. Dean, M. Seiffert, F. Putz, L. Becker, and D. R. Thomas, “A Data-Driven Evaluation of the Current Security State of Android Devices,” in *IEEE Conference on Communications and Network Security (CNS) 2024*, 2024, pp. 1–9.
 - [35] Google, “Pixel security,” Online, 2025, accessed: 2025-09-16. [Online]. Available: <https://safety.google/pixel/>
 - [36] European Commission, “Draft implementing regulation on European Digital Identity Wallets certification,” European Commission, Tech. Rep. Ares(2024)5786790, 2024, initiative 14337. [Online]. Available: https://ec.europa.eu/info/law/better-regulation/have-your-say/initiatives/14337-European-Digital-Identity-Wallets-certification_en
 - [37] “Common Criteria for Information Technology Security Evaluation, Part 5,” Common Criteria Development Board, CC:2022, Release 1, 2022, accessed: 2025-09-11. [Online]. Available: <https://www.commoncriteriaportal.org/files/ccfiles/CC2022PART5R1.pdf>
 - [38] T. B.V., “Certificate: H1D3 secure microcontroller with crypto library v1.3.10,” TrustCB B.V., Tech. Rep., oct 2023. [Online]. Available: <https://www.commoncriteriaportal.org/nfs/ccpfiles/files/epfiles/NSCIB-CC-2300073-02-CR.pdf>
 - [39] eu-digital-identity-wallet GitHub organization, “EUDI Architecture and Reference Framework,” GitHub release v2.4.0, 2025, accessed: 2025-09-05. [Online]. Available: <https://github.com/eu-digital-identity-wallet/eudi-doc-architecture-and-reference-framework/releases/tag/v2.4.0>
 - [40] D. Cater, “Certification Report: Qualcomm® Trusted Execution Environment (TEE) v5.8 on Qualcomm® Snapdragon™ 865,” TÜV Rheinland Nederland B.V., Tech. Rep., aug 2021. [Online]. Available: <https://www.commoncriteriaportal.org/nfs/ccpfiles/files/epfiles/NSCIB-CC-0244671-CR-1.0.pdf>
 - [41] T. Sušánka and J. Kokeš, “Security Analysis of the Telegram IM,” in *Reversing and Offensive-Oriented Trends Symposium (ROOTS) 2017*. Association for Computing Machinery, 2017. [Online]. Available: <https://doi.org/10.1145/3150376.3150382>
 - [42] L. Chen, D. Moody, A. Regenscheid, A. Robinson, and K. Randall, “Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication (SP) 800-186, 2023.
 - [43] P. L. Montgomery, “Speeding the Pollard and elliptic curve methods of factorization,” *Mathematics of Computation*, vol. 48, no. 177, pp. 243–264, 1987.
 - [44] L. Batina, Ł. Chmielewski, L. Papachristodoulou, P. Schwabe, and M. Tunstall, “Online template attacks,” *Journal of Cryptographic Engineering*, vol. 9, no. 1, pp. 21–36, 2019. [Online]. Available: <https://doi.org/10.1007/s13389-017-0171-8>
 - [45] M. Dugardin, L. Papachristodoulou, Z. Najm, L. Batina, J.-L. Danger, and S. Guilley, “Dismantling Real-World ECC with Horizontal and Vertical Tem-

- plate Attacks,” in *Constructive Side-Channel Analysis and Secure Design*. Springer, 2016, pp. 88–108.
- [46] N. Roelofs, N. Samwel, L. Batina, and J. Daemen, “Online Template Attack on ECDSA: Extracting Keys via the Other Side,” in *AFRICACRYPT 2020*. Springer, 2020, pp. 323–336. [Online]. Available: https://doi.org/10.1007/978-3-030-51938-4_16
- [47] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security Signatures,” in *CHES 2011*. Springer, 2011, pp. 124–142.
- [48] L. Weissbart, S. Picek, and L. Batina, “One Trace Is All It Takes: Machine Learning-Based Side-Channel Attack on EdDSA,” in *SPACE 2019*, ser. Lecture Notes in Computer Science, vol. 11947. Springer, 2019, pp. 86–105. [Online]. Available: https://doi.org/10.1007/978-3-030-35869-3_8
- [49] S. Jin, S. Lee, S. M. Cho, H. Kim, and S. Hong, “Novel Key Recovery Attack on Secure ECDSA Implementation by Exploiting Collisions between Unknown Entries,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 4, pp. 1–26, 2021. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/9058>
- [50] M. Medwed and E. Oswald, “Template Attacks on ECDSA,” in *Information Security Applications*. Springer, 2009, pp. 14–27.
- [51] T. Izu and T. Takagi, “A Fast Parallel Elliptic Curve Multiplication Resistant against Side Channel Attacks,” in *Public Key Cryptography (PKC) 2002*, ser. Lecture Notes in Computer Science, vol. 2274. Springer, 2002, pp. 280–296. [Online]. Available: https://doi.org/10.1007/3-540-45664-3_20
- [52] Y. Nakano, Y. Souissi, R. Nguyen, L. Sauvage, J.-L. Danger, S. Guilley, S. Kiyomoto, and Y. Miyake, “A Pre-processing Composition for Secret Key Recovery on Android Smartphone,” in *IFIP WISTP 2014*, ser. Lecture Notes in Computer Science, vol. 8501. Springer, 2014, pp. 76–91. [Online]. Available: https://doi.org/10.1007/978-3-662-43826-8_6
- [53] R. Callan, A. Zajić, and M. Prvulovic, “FASE: finding amplitude-modulated side-channel emanations,” in *International Symposium on Computer Architecture (ISCA) 2015*. Association for Computing Machinery, 2015, pp. 592–603. [Online]. Available: <https://doi.org/10.1145/2749469.2750394>
- [54] M. Prvulovic, A. Zajić, R. L. Callan, and C. J. Wang, “A Method for Finding Frequency-Modulated and Amplitude-Modulated Electromagnetic Emanations in Computer Systems,” *IEEE Transactions on Electromagnetic Compatibility*, vol. 59, no. 1, pp. 34–42, 2017.
- [55] C. Wang, R. Callan, A. Zajić, and M. Prvulovic, “An algorithm for finding carriers of amplitude-modulated electromagnetic emanations in computer systems,” in *10th European Conference on Antennas and Propagation (EuCAP) 2016*, 2016.
- [56] B. B. Yilmaz, R. L. Callan, M. Prvulovic, and A. Zajić, “Capacity of the EM Covert/Side-Channel Created by the Execution of Instructions in a Processor,” *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 3, pp. 605–620, 2018.
- [57] K. A. Vedros, C. Kolias, D. Barbara, and R. C. Ivans, “From Code to EM Signals: A Generative Approach to Side Channel Analysis-based Anomaly Detection,” in *International Conference on Availability, Reliability and Security (ARES) 2024*. Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3664476.3664520>
- [58] T. Schneider and A. Moradi, “Leakage Assessment Methodology - A Clear Roadmap for Side-Channel Evaluations,” in *CHES 2015*, ser. Lecture Notes in Computer Science, vol. 9293. Springer, 2015, pp. 495–513. [Online]. Available: https://doi.org/10.1007/978-3-662-48324-4_25
- [59] A. Gohr, S. Jacob, and W. Schindler, “Efficient Solutions of the CHES 2018 AES Challenge Using Deep Residual Neural Networks and Knowledge Distillation on Adversarial Examples,” *IACR Cryptology ePrint Archive*, 2020. [Online]. Available: <https://eprint.iacr.org/2020/165>
- [60] S. Picek, G. Perin, L. Mariot, L. Wu, and L. Batina, “SoK: Deep Learning-based Physical Side-channel Analysis,” *ACM Computing Surveys*, vol. 55, no. 11, pp. 227:1–227:35, 2023. [Online]. Available: <https://doi.org/10.1145/3569577>
- [61] R. Benadjila, E. Prouff, R. Strullu, E. Cagli, and C. Dumas, “Deep learning for side-channel analysis and introduction to ASCAD database,” *Journal of Cryptographic Engineering*, vol. 10, 2020.
- [62] “Qualcomm Snapdragon 750G 5G Mobile Platform Product Brief,” Qualcomm Technologies, Inc., Tech. Rep., 2020, accessed: 2025-08. [Online]. Available: https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/snapdragon_750g_5g_mobile_platform_product_brief_0.pdf
- [63] J. L. Hintze and R. D. Nelson, “Violin Plots: A Box Plot-Density Trace Synergism,” *The American Statistician*, vol. 52, no. 2, pp. 181–184, 1998.
- [64] D. Boneh and R. Venkatesan, “Hardness of Computing the Most Significant Bits of Secret Keys in Diffie-Hellman and Related Schemes,” in *Advances in Cryptology - CRYPTO '96*, ser. Lecture Notes in Computer Science, vol. 1109. Springer, 1996, pp. 129–142.
- [65] D. Bleichenbacher, “On the generation of one-time keys in DL signature schemes,” 2000, presentation at IEEE P1363 working group meeting.
- [66] D. F. Aranha, F. R. Novaes, A. Takahashi, M. Tibouchi, and Y. Yarom, “LadderLeak: Breaking ECDSA with Less than One Bit of Nonce Leakage,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS) 2020*. Association for Computing Machinery, 2020, pp. 225–242. [Online]. Available: <https://doi.org/10.1145/3372297.3417268>
- [67] E. D. Mulder, M. Hutter, M. E. Marson, and P. Pearson, “Using Bleichenbacher’s Solution to the Hidden Number Problem to Attack Nonce Leaks in 384-Bit ECDSA,” in *CHES 2013*. Springer, 2013, pp. 435–452. [Online]. Available: https://doi.org/10.1007/978-3-642-40349-1_25
- [68] M. R. Albrecht and N. Heninger, “On Bounded Distance Decoding with Predicate: Breaking the ‘Lattice Barrier’ for the Hidden Number Problem,” in *Advances in Cryptology - EUROCRYPT 2021*. Springer, 2021, pp. 528–558. [Online]. Available:

- https://doi.org/10.1007/978-3-030-77870-5_19
- [69] C. Sun, T. Espitau, M. Tibouchi, and M. Abe, “Guessing Bits: Improved Lattice Attacks on (EC)DSA with Nonce Leakage,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 1, pp. 391–413, 2021. [Online]. Available: <https://tosc.iacr.org/index.php/TCHEs/article/view/9302>
 - [70] L. Xu, Z. Dai, B. Wu, and D. Lin, “Improved Attacks on (EC)DSA with Nonce Leakage by Lattice Sieving with Predicate,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 2, pp. 568–586, 2023. [Online]. Available: <https://tches.iacr.org/index.php/TCHEs/article/view/10294>
 - [71] Y. Gao, J. Wang, H. Hu, and B. He, “Attacking ECDSA with Nonce Leakage by Lattice Sieving: Bridging the Gap with Fourier Analysis-Based Attacks,” in *Advances in Cryptology – ASIACRYPT 2024*. Springer, 2024, pp. 3–34. [Online]. Available: https://doi.org/10.1007/978-981-96-0944-4_1
 - [72] European Union Agency for Cybersecurity (ENISA), “EUCC Scheme State-of-the-Art Document: Application of Attack Potential to Smartcards and Similar Devices,” European Union Agency for Cybersecurity (ENISA), Version 2, Feb. 2025, endorsed for publication on 11/03/2025 by the European Cybersecurity Certification Group (ECCG). [Online]. Available: https://certification.enisa.europa.eu/index_en
 - [73] L. Batina, L. Chmielewski, B. Haase, N. Samwel, and P. Schwabe, “SoK: SCA-secure ECC in Software - Mission Impossible?” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 1, pp. 557–589, 2023. [Online]. Available: <https://doi.org/10.46586/tches.v2023.i1.557-589>
 - [74] K. Itoh, T. Izu, and M. Takenaka, “A Practical Countermeasure against Address-Bit Differential Power Analysis,” in *CHES 2003*. Springer, 2003, pp. 382–396.
 - [75] J. Heyszl, S. Mangard, B. Heinz, F. Stumpf, and G. Sigl, “Localized Electromagnetic Analysis of Cryptographic Implementations,” in *Topics in Cryptology – CT-RSA 2012*. Springer, 2012, pp. 231–244.
 - [76] J.-S. Coron, “Resistance Against Differential Power Analysis For Elliptic Curve Cryptosystems,” in *CHES 1999*. Springer, 1999, pp. 292–302.
 - [77] S. Chari, J. R. Rao, and P. Rohatgi, “Template Attacks,” in *CHES 2002*, ser. Lecture Notes in Computer Science, vol. 2523. Springer, 2002, pp. 13–28. [Online]. Available: https://doi.org/10.1007/3-540-36400-5_3
 - [78] C. Rechberger and E. Oswald, “Practical Template Attacks,” in *WISA 2004*, ser. Lecture Notes in Computer Science, vol. 3325. Springer, 2004, pp. 440–456. [Online]. Available: https://doi.org/10.1007/978-3-540-31815-6_35



(a) The Raspberry Pi 4's (b) IR die-shot of the BCM2711 chip with removed BCM2711 SoC. The four metal lid. On the right side of ARM Cortex-A72 cores are the die, six capacitors are visible. outlined with blue rectangles.

Figure 11: Raspberry Pi 4 Model B Target

Appendix A. Acknowledgment

This research was supported by the German Federal Office for Information Security (BSI) and the Bavarian Ministry of Economic Affairs, Regional Development and Energy in the context of the project Trusted Electronics Bavaria (TrEB).

Appendix B. Disclosure

We adhered to a coordinated disclosure process for this work. In October 2025, we notified the OpenSSL and libgcrypt security teams about the demonstrated side-channel leakage in their ECDSA implementations and provided our assessment of existing countermeasures. Please note that while both libraries may choose to integrate countermeasures against physical side-channel attacks, they do not consider such attacks in their threat models^{17,18}.

At the same time we informed Broadcom and the Raspberry Pi Foundation as well as Qualcomm and Fairphone about our empirical findings on the exemplary Raspberry Pi 4 (BCM2711 SoC) and Fairphone 4 (Snapdragon 750G 5G SoC) targets, including our threat models, required lab setup, and the distinction between invasive and non-invasive measurement scenarios.

None of the contacted entities voiced concerns about this publication.

Appendix C. Raspberry Pi 4 and the BCM2711 SoC

Fig. 11a shows the Broadcom BCM2711 SoC with removed metal lid. We recorded an IR die-shot to determine the location of the four CPUs for localized measurements on the die (Fig. 11b). The die-shot was acquired with a WIDY SWIR 640V-S camera from New Imaging Technologies.

17. <https://openssl-library.org/policies/general/security-policy/>

18. <https://www.gnupg.org/documentation/security.html>

Appendix D.

Montgomery ladder step in OpenSSL

OpenSSL uses Algorithm 5 for the step operation of the Montgomery ladder. Each odd line in Algorithm 5 contains finite-field multiplication and square operations. The even lines contain addition, subtraction or shift operations (used to implement the multiplication with a power of two), which are comparatively short. The operations can be clearly mapped to the 5-2-1-2-3-1-3-3 pattern of blocks in the bandpass-filtered signal and adjacent peaks in the bandpass + absolute + sliding-median filtered signal of Fig. 1. Each multiplication/square operation leads to a block/peak and the operations on the even lines make up the breaks between these blocks/peaks. For example, line 1 corresponds to the 5 adjacent peaks at the beginning of the trace in Fig. 1.

Algorithm 5 Montgomery ladder step in OpenSSL with differential addition-and-doubling formulas from [51].

Input:

Weierstrass equation: $y^2 \equiv x^3 + ax + b \pmod{p}$;
Input points in projective coordinates: $s = (X_1 : Y_1 : Z_1)$,
 $r = (X_2 : Y_2 : Z_2)$;
Generator point in affine coordinates: $p = (x, y)$.

Output:

Result in s, r as projective coordinates: $s \leftarrow r + s$, $r \leftarrow 2r$.

- 1: $t_6 \leftarrow X_2 \cdot X_1$; $t_0 \leftarrow Z_2 \cdot Z_1$; $t_4 \leftarrow X_2 \cdot Z_1$; $t_3 \leftarrow Z_2 \cdot X_1$; $t_5 \leftarrow a \cdot t_0$
- 2: $t_4 \leftarrow t_6 + t_5$; $t_6 \leftarrow t_3 + t_4$
- 3: $t_5 \leftarrow t_6 \cdot t_5$; $t_0 \leftarrow t_0^2$
- 4: $t_2 \leftarrow b \ll 2$
- 5: $t_0 \leftarrow t_2 \cdot t_0$
- 6: $t_5 \leftarrow t_5 \ll 1$; $t_3 \leftarrow t_4 - t_3$
- 7: $Z_1 \leftarrow t_3^2$; $t_4 \leftarrow Z_1 \cdot x$
- 8: $t_0 \leftarrow t_0 + t_5$; $X_1 \leftarrow t_0 - t_4$
- 9: $t_4 \leftarrow X_2^2$; $t_5 \leftarrow Z_2^2$; $t_6 \leftarrow a \cdot t_5$
- 10: $t_1 \leftarrow X_2 + Z_2$
- 11: $t_1 \leftarrow t_1^2$
- 12: $t_1 \leftarrow t_1 - t_4$; $t_1 \leftarrow t_1 - t_5$; $t_3 \leftarrow t_4 - t_6$
- 13: $t_3 \leftarrow t_3^2$; $t_0 \leftarrow t_5 \cdot t_1$; $t_0 \leftarrow t_2 \cdot t_0$
- 14: $X_2 \leftarrow t_3 - t_0$; $t_3 \leftarrow t_4 + t_6$
- 15: $t_4 \leftarrow t_5^2$; $t_4 \leftarrow t_4 \cdot t_2$; $t_1 \leftarrow t_1 \cdot t_3$
- 16: $t_1 \leftarrow t_1 \ll 1$; $Z_2 \leftarrow t_4 + t_1$

Appendix E.

Evaluation of classifiers for swap conditions

Fig. 12 shows the accuracy of multiple classifiers. The ASCAD networks were taken from commit id 410b92b¹⁹. ASCAD includes CNN_1 and CNN_2 , which differ by the fact that CNN_2 uses a stride length of two for the convolution layer in the first block. Our results are for CNN_1 . Since those networks were designed for predicting a byte within the AES state, we also conducted a short hyper parameter search, which led to the architecture described in Table 2. At a success rate of 98.14%, this custom CNN performs marginally better than the residual neural

¹⁹. <https://github.com/ANSSI-FR/ASCAD/tree/410b92bab3bc69502b43c5cc9ccdec74794870be>

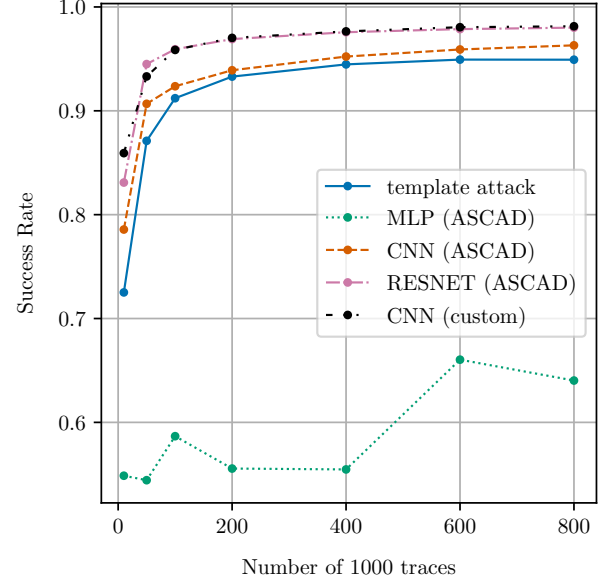


Figure 12: Success rate of various classifiers in relation to number of training traces.

TABLE 2: Architecture of the swap condition classifier CNN.

Layer	Description
Input	Input of shape (1000, 1)
Batch Normalization	
Conv1D	32 filters, kernel size 90, ReLU
Conv1D	32 filters, kernel size 90, ReLU
Average Pooling	Pool size=2, strides=2
Conv1D	64 filters, kernel size 90, ReLU
Conv1D	64 filters, kernel size 90, ReLU
Average Pooling	Pool size=2, strides=2
... (3 more blocks with doubling of filters) ...	
Flatten	
Dense	Fully connected, 4096 neurons, ReLU
Dense (Output)	Fully connected, 2 neurons, softmax

network taken from the ASCAD project. At an accuracy of 94.92%, the Gaussian template attack [77, 78] is outperformed by all networks but the multilayer perceptron (MLP).

Appendix F.

Snapdragon 750G 5G SoC

Fig. 13 shows an IR die-shot of the SoC. The SoC includes a Kryo 570 octa-core CPU [62]. Due to their distinct structures, the two A77 and six A55 cores can be identified. By installing postmarketOS and using the *cpufreq-info* tool, we obtained the supported frequencies. The Cortex-A55 CPUs can be clocked at the following frequencies:

- 300 MHz
- 576 MHz
- 768 MHz
- 1.02 GHz
- 1.25 GHz
- 1.32 GHz
- 1.52 GHz
- 1.61 GHz
- 1.71 GHz
- 1.80 GHz



Figure 13: IR die-shot of the Snapdragon 750G 5G SoC. The two ARM Cortex-A77 cores are outlined in blue, the six ARM Cortex-A55 cores are marked with yellow rectangles.

The Cortex-A77 CPUs can be clocked at:

- 300 MHz
- 787 MHz
- 979 MHz
- 1.04 GHz
- 1.25 GHz
- 1.40 GHz
- 1.56 GHz
- 1.77 GHz
- 1.90 GHz
- 2.07 GHz
- 2.13 GHz
- 2.21 GHz